

12.2.2 Hypothesis Space Search

How can the domain theory and training data best be combined to constrain the search for an acceptable hypothesis? This remains an open question in machine learning. This chapter surveys a variety of approaches that have been proposed, many of which consist of extensions to inductive methods we have already studied (e.g., BACKPROPAGATION, FOIL).

One way to understand the range of possible approaches is to return to our view of learning as a task of searching through the space of alternative hypotheses. We can characterize most learning methods as search algorithms by describing the hypothesis space H they search, the initial hypothesis h_0 at which they begin their search, the set of search operators O that define individual search steps, and the goal criterion G that specifies the search objective. In this chapter we explore three different methods for using prior knowledge to alter the search performed by purely inductive methods.

- *Use prior knowledge to derive an initial hypothesis from which to begin the search.* In this approach the domain theory B is used to construct an initial hypothesis h_0 that is consistent with B . A standard inductive method is then applied, starting with the initial hypothesis h_0 . For example, the KBANN system described below learns artificial neural networks in this way. It uses prior knowledge to design the interconnections and weights for an initial network, so that this initial network is perfectly consistent with the given domain theory. This initial network hypothesis is then refined inductively using the BACKPROPAGATION algorithm and available data. Beginning the search at a hypothesis consistent with the domain theory makes it more likely that the final output hypothesis will better fit this theory.
- *Use prior knowledge to alter the objective of the hypothesis space search.* In this approach, the goal criterion G is modified to require that the output hypothesis fits the domain theory as well as the training examples. For example, the EBNN system described below learns neural networks in this way. Whereas inductive learning of neural networks performs gradient descent search to minimize the squared error of the network over the training data, EBNN performs gradient descent to optimize a different criterion. This modified criterion includes an additional term that measures the error of the learned network relative to the domain theory.
- *Use prior knowledge to alter the available search steps.* In this approach, the set of search operators O is altered by the domain theory. For example, the FOCL system described below learns sets of Horn clauses in this way. It is based on the inductive system FOIL, which conducts a greedy search through the space of possible Horn clauses, at each step revising its current hypothesis by adding a single new literal. FOCL uses the domain theory to expand the set of alternatives available when revising the hypothesis, allowing the

addition of multiple literals in a single search step when warranted by the domain theory. In this way, FOCL allows single-step moves through the hypothesis space that would correspond to many steps using the original inductive algorithm. These “macro-moves” can dramatically alter the course of the search, so that the final hypothesis found consistent with the data is different from the one that would be found using only the inductive search steps.

The following sections describe each of these approaches in turn.

12.3 USING PRIOR KNOWLEDGE TO INITIALIZE THE HYPOTHESIS

One approach to using prior knowledge is to initialize the hypothesis to perfectly fit the domain theory, then inductively refine this initial hypothesis as needed to fit the training data. This approach is used by the KBANN (Knowledge-Based Artificial Neural Network) algorithm to learn artificial neural networks. In KBANN an initial network is first constructed so that for every possible instance, the classification assigned by the network is identical to that assigned by the domain theory. The BACKPROPAGATION algorithm is then employed to adjust the weights of this initial network as needed to fit the training examples.

It is easy to see the motivation for this technique: if the domain theory is correct, the initial hypothesis will correctly classify all the training examples and there will be no need to revise it. However, if the initial hypothesis is found to imperfectly classify the training examples, then it will be refined inductively to improve its fit to the training examples. Recall that in the purely inductive BACKPROPAGATION algorithm, weights are typically initialized to small random values. The intuition behind KBANN is that even if the domain theory is only approximately correct, initializing the network to fit this domain theory will give a better starting approximation to the target function than initializing the network to random initial weights. This should lead, in turn, to better generalization accuracy for the final hypothesis.

This *initialize-the-hypothesis* approach to using the domain theory has been explored by several researchers, including Shavlik and Towell (1989), Towell and Shavlik (1994), Fu (1989, 1993), and Pratt (1993a, 1993b). We will use the KBANN algorithm described in Shavlik and Towell (1989) to illustrate this approach.

12.3.1 The KBANN Algorithm

The KBANN algorithm exemplifies the initialize-the-hypothesis approach to using domain theories. It assumes a domain theory represented by a set of propositional, nonrecursive Horn clauses. A Horn clause is propositional if it contains no variables. The input and output of KBANN are as follows:

KBANN(*Domain_Theory*, *Training_Examples*)

Domain_Theory: Set of propositional, nonrecursive Horn clauses.

Training_Examples: Set of (input output) pairs of the target function.

Analytical step: Create an initial network equivalent to the domain theory.

1. For each instance attribute create a network input.
2. For each Horn clause in the *Domain_Theory*, create a network unit as follows:
 - Connect the inputs of this unit to the attributes tested by the clause antecedents.
 - For each non-negated antecedent of the clause, assign a weight of W to the corresponding sigmoid unit input.
 - For each negated antecedent of the clause, assign a weight of $-W$ to the corresponding sigmoid unit input.
 - Set the threshold weight w_0 for this unit to $-(n - .5)W$, where n is the number of non-negated antecedents of the clause.
3. Add additional connections among the network units, connecting each network unit at depth i from the input layer to all network units at depth $i + 1$. Assign random near-zero weights to these additional connections.

Inductive step: Refine the initial network.

4. Apply the BACKPROPAGATION algorithm to adjust the initial network weights to fit the *Training_Examples*.
-

TABLE 12.2

The KBANN algorithm. The domain theory is translated into an equivalent neural network (steps 1–3), which is inductively refined using the BACKPROPAGATION algorithm (step 4). A typical value for the constant W is 4.0.

Given:

- A set of training examples
- A domain theory consisting of nonrecursive, propositional Horn clauses

Determine:

- An artificial neural network that fits the training examples, biased by the domain theory

The two stages of the KBANN algorithm are first to create an artificial neural network that perfectly fits the domain theory and second to use the BACKPROPAGATION algorithm to refine this initial network to fit the training examples. The details of this algorithm, including the algorithm for creating the initial network, are given in Table 12.2 and illustrated in Section 12.3.2.

12.3.2 An Illustrative Example

To illustrate the operation of KBANN, consider the simple learning problem summarized in Table 12.3, adapted from Towell and Shavlik (1989). Here each instance describes a physical object in terms of the material from which it is made, whether it is light, etc. The task is to learn the target concept *Cup* defined over such physical objects. Table 12.3 describes a set of training examples and a domain theory for the *Cup* target concept. Notice the domain theory defines a *Cup*

Domain theory:

Cup $\leftarrow$ *Stable, Lifiable, OpenVessel*
Stable $\leftarrow$ *BottomIsFlat*
Lifiable $\leftarrow$ *Graspable, Light*
Graspable $\leftarrow$ *HasHandle*
OpenVessel $\leftarrow$ *HasConcavity, ConcavityPointsUp*

Training examples:

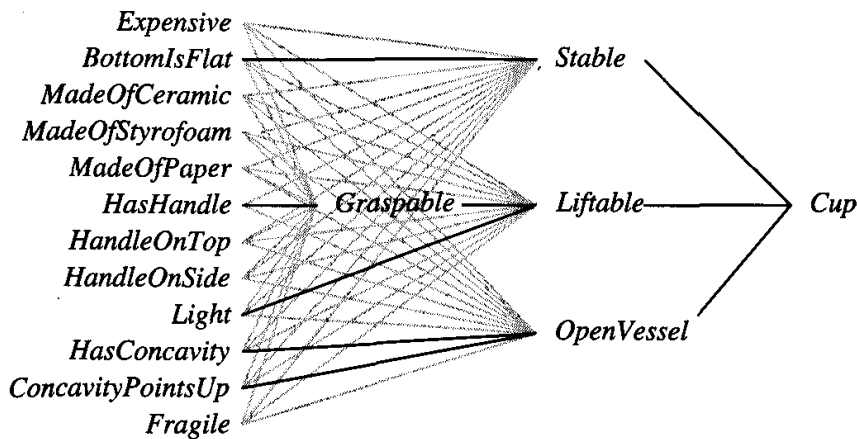
	<i>Cups</i>				<i>Non-Cups</i>					
<i>BottomIsFlat</i>	✓	✓	✓	✓	✓	✓	✓			✓
<i>ConcavityPointsUp</i>	✓	✓	✓	✓	✓		✓	✓		
<i>Expensive</i>	✓		✓				✓		✓	
<i>Fragile</i>	✓	✓			✓	✓		✓		✓
<i>HandleOnTop</i>					✓		✓			
<i>HandleOnSide</i>	✓			✓					✓	
<i>HasConcavity</i>	✓	✓	✓	✓	✓		✓	✓	✓	✓
<i>HasHandle</i>	✓			✓	✓		✓		✓	
<i>Light</i>	✓	✓	✓	✓	✓	✓	✓		✓	
<i>MadeOfCeramic</i>	✓				✓		✓	✓		
<i>MadeOfPaper</i>				✓					✓	
<i>MadeOfStyrofoam</i>		✓	✓		✓					✓

TABLE 12.3

The *Cup* learning task. An approximate domain theory and a set of training examples for the target concept *Cup*.

as an object that is *Stable*, *Lifiable*, and an *OpenVessel*. The domain theory also defines each of these three attributes in terms of more primitive attributes, terminating in the primitive, operational attributes that describe the instances. Note the domain theory is not perfectly consistent with the training examples. For example, the domain theory fails to classify the second and third training examples as positive examples. Nevertheless, the domain theory forms a useful approximation to the target concept. KBANN uses the domain theory and training examples together to learn the target concept more accurately than it could from either alone.

In the first stage of the KBANN algorithm (steps 1–3 in the algorithm), an initial network is constructed that is consistent with the domain theory. For example, the network constructed from the *Cup* domain theory is shown in Figure 12.2. In general the network is constructed by creating a sigmoid threshold unit for each Horn clause in the domain theory. KBANN follows the convention that a sigmoid output value greater than 0.5 is interpreted as *True* and a value below 0.5 as *False*. Each unit is therefore constructed so that its output will be greater than 0.5 just in those cases where the corresponding Horn clause applies. For each antecedent to the Horn clause, an input is created to the corresponding sigmoid unit. The weights of the sigmoid unit are then set so that it computes the logical AND of its inputs. In particular, for each input corresponding to a non-negated antecedent,

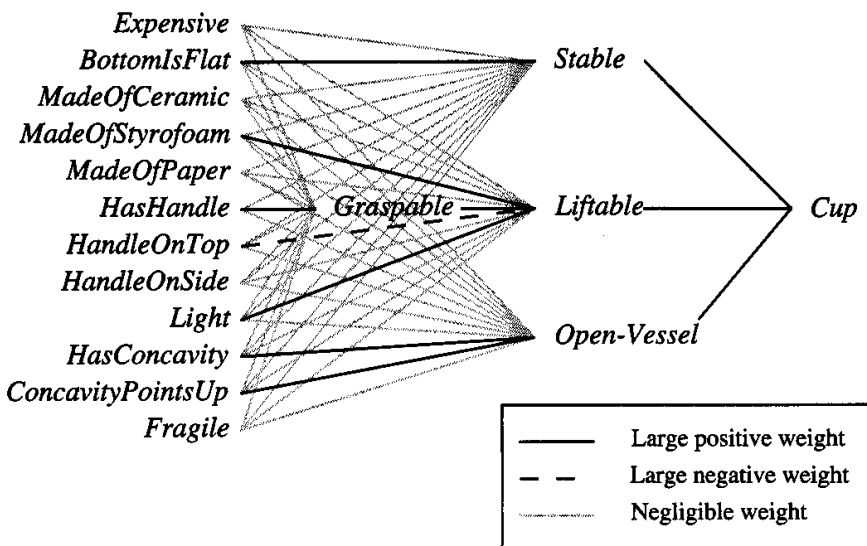
**FIGURE 12.2**

A neural network equivalent to the domain theory. This network, created in the first stage of the KBANN algorithm, produces output classifications identical to those of the given domain theory clauses. Dark lines indicate connections with weight W and correspond to antecedents of clauses from the domain theory. Light lines indicate connections with weights of approximately zero.

the weight is set to some positive constant W . For each input corresponding to a negated antecedent, the weight is set to $-W$. The threshold weight of the unit, w_0 is then set to $-(n - .5)W$, where n is the number of non-negated antecedents. When unit input values are 1 or 0, this assures that their weighted sum plus w_0 will be positive (and the sigmoid output will therefore be greater than 0.5) if and only if all clause antecedents are satisfied. Note for sigmoid units at the second and subsequent layers, unit inputs will not necessarily be 1 and 0 and the above argument may not apply. However, if a sufficiently large value is chosen for W , this KBANN algorithm can correctly encode the domain theory for arbitrarily deep networks. Towell and Shavlik (1994) report using $W = 4.0$ in many of their experiments.

Each sigmoid unit input is connected to the appropriate network input or to the output of another sigmoid unit, to mirror the graph of dependencies among the corresponding attributes in the domain theory. As a final step many additional inputs are added to each threshold unit, with their weights set approximately to zero. The role of these additional connections is to enable the network to inductively learn additional dependencies beyond those suggested by the given domain theory. The solid lines in the network of Figure 12.2 indicate unit inputs with weights of W , whereas the lightly shaded lines indicate connections with initial weights near zero. It is easy to verify that for sufficiently large values of W this network will output values identical to the predictions of the domain theory.

The second stage of KBANN (step 4 in the algorithm of Table 12.2) uses the training examples and the BACKPROPAGATION algorithm to refine the initial network weights. Of course if the domain theory and training examples contain no errors, the initial network will already fit the training data. In the *Cup* example, however, the domain theory and training data are inconsistent, and this step therefore alters the initial network weights. The resulting trained network is summarized in Figure 12.3, with dark solid lines indicating the largest positive weights, dashed lines indicating the largest negative weights, and light lines

**FIGURE 12.3**

Result of inductively refining the initial network. KBANN uses the training examples to modify the network weights derived from the domain theory. Notice the new dependency of *Liftable* on *MadeOfStyrofoam* and *HandleOnTop*.

indicating negligible weights. Although the initial network misclassifies several training examples from Table 12.3, the refined network of Figure 12.3 perfectly classifies all of these training examples.

It is interesting to compare the final, inductively refined network weights to the initial weights derived from the domain theory. As can be seen in Figure 12.3, significant new dependencies were discovered during the inductive step, including the dependency of the *Liftable* unit on the feature *MadeOfStyrofoam*. It is important to keep in mind that while the unit labeled *Liftable* was initially defined by the given Horn clause for *Liftable*, the subsequent weight changes performed by BACKPROPAGATION may have dramatically changed the meaning of this hidden unit. After training of the network, this unit may take on a very different meaning unrelated to the initial notion of *Liftable*.

12.3.3 Remarks

To summarize, KBANN analytically creates a network equivalent to the given domain theory, then inductively refines this initial hypothesis to better fit the training data. In doing so, it modifies the network weights as needed to overcome inconsistencies between the domain theory and observed data.

The chief benefit of KBANN over purely inductive BACKPROPAGATION (beginning with random initial weights) is that it typically generalizes more accurately than BACKPROPAGATION when given an approximately correct domain theory, especially when training data is scarce. KBANN and other initialize-the-hypothesis approaches have been demonstrated to outperform purely inductive systems in several practical problems. For example, Towell et al. (1990) describe the application of KBANN to a molecular genetics problem. Here the task was to learn to

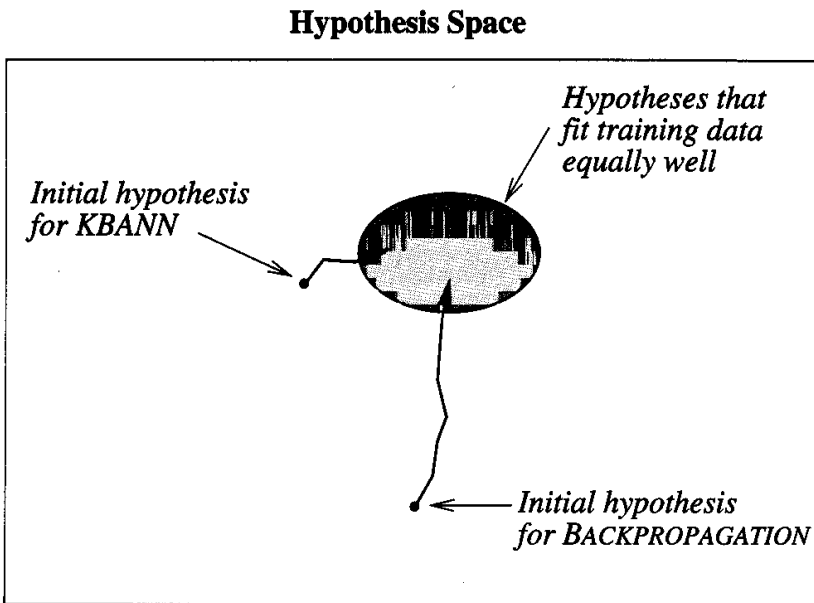
recognize DNA segments called promoter regions, which influence gene activity. In this experiment KBANN was given an initial domain theory obtained from a molecular geneticist, and a set of 53 positive and 53 negative training examples of promoter regions. Performance was evaluated using a leave-one-out strategy in which the system was run 106 different times. On each iteration KBANN was trained using 105 of the 106 examples and tested on the remaining example. The results of these 106 experiments were accumulated to provide an estimate of the true error rate. KBANN obtained an error rate of 4/106, compared to an error rate of 8/106 using standard BACKPROPAGATION. A variant of the KBANN approach was applied by Fu (1993), who reports an error rate of 2/106 on the same data. Thus, the impact of prior knowledge in these experiments was to reduce significantly the error rate. The training data for this experiment is available at World Wide Web site <http://www.ics.uci.edu/~mlearn/MLRepository.html>.

Both Fu (1993) and Towell et al. (1990) report that Horn clauses extracted from the final trained network provided a refined domain theory that better fit the observed data. Although it is sometimes possible to map from the learned network weights back to a refined set of Horn clauses, in the general case this is problematic because some weight settings have no direct Horn clause analog. Craven and Shavlik (1994) and Craven (1996) describe alternative methods for extracting symbolic rules from learned networks.

To understand the significance of KBANN it is useful to consider how its hypothesis search differs from that of the purely inductive BACKPROPAGATION algorithm. The hypothesis space search conducted by both algorithms is depicted schematically in Figure 12.4. As shown there, the key difference is the initial hypothesis from which weight tuning is performed. In the case that multiple hypotheses (weight vectors) can be found that fit the data—a condition that will be especially likely when training data is scarce—KBANN is likely to converge to a hypothesis that generalizes beyond the data in a way that is more similar to the domain theory predictions. On the other hand, the particular hypothesis to which BACKPROPAGATION converges will more likely be a hypothesis with small weights, corresponding roughly to a generalization bias of smoothly interpolating between training examples. In brief, KBANN uses a domain-specific theory to bias generalization, whereas BACKPROPAGATION uses a domain-independent syntactic bias toward small weight values. Note in this summary we have ignored the effect of local minima on the search.

Limitations of KBANN include the fact that it can accommodate only propositional domain theories; that is, collections of variable-free Horn clauses. It is also possible for KBANN to be misled when given highly inaccurate domain theories, so that its generalization accuracy can deteriorate below the level of BACKPROPAGATION. Nevertheless, it and related algorithms have been shown to be useful for several practical problems.

KBANN illustrates the initialize-the-hypothesis approach to combining analytical and inductive learning. Other examples of this approach include Fu (1993); Gallant (1988); Bradshaw et al. (1989); Yang and Bhargava (1990); Lacher et al. (1991). These approaches vary in the exact technique for constructing the initial

**FIGURE 12.4**

Hypothesis space search in KBANN. KBANN initializes the network to fit the domain theory, whereas BACKPROPAGATION initializes the network to small random weights. Both then refine the weights iteratively using the same gradient descent rule. When multiple hypotheses can be found that fit the training data (shaded region), KBANN and BACKPROPAGATION are likely to find different hypotheses due to their different starting points.

network, the application of BACKPROPAGATION to weight tuning, and in methods for extracting symbolic descriptions from the refined network. Pratt (1993a, 1993b) describes an initialize-the-hypothesis approach in which the prior knowledge is provided by a previously learned neural network for a related task, rather than a manually provided symbolic domain theory. Methods for training the values of Bayesian belief networks, as discussed in Section 6.11, can also be viewed as using prior knowledge to initialize the hypothesis. Here the prior knowledge corresponds to a set of conditional independence assumptions that determine the graph structure of the Bayes net, whose conditional probability tables are then induced from the training data.

12.4 USING PRIOR KNOWLEDGE TO ALTER THE SEARCH OBJECTIVE

The above approach begins the gradient descent search with a hypothesis that perfectly fits the domain theory, then perturbs this hypothesis as needed to maximize the fit to the training data. An alternative way of using prior knowledge is to incorporate it into the error criterion minimized by gradient descent, so that the network must fit a combined function of the training data and domain theory. In this section, we consider using prior knowledge in this fashion. In particular, we consider prior knowledge in the form of known derivatives of the target function. Certain types of prior knowledge can be expressed quite naturally in this form. For example, in training a neural network to recognize handwritten characters we

can specify certain derivatives of the target function in order to express our prior knowledge that “the identity of the character is independent of small translations and rotations of the image.”

Below we describe the TANGENTPROP algorithm, which trains a neural network to fit both training values and training derivatives. Section 12.4.4 then describes how these training derivatives can be obtained from a domain theory similar to the one used in the *Cup* example of Section 12.3. In particular, it discusses how the EBNN algorithm constructs explanations of individual training examples in order to extract training derivatives for use by TANGENTPROP. TANGENTPROP and EBNN have been demonstrated to outperform purely inductive methods in a variety of domains, including character and object recognition, and robot perception and control tasks.

12.4.1 The TANGENTPROP Algorithm

TANGENTPROP (Simard et al. 1992) accommodates domain knowledge expressed as derivatives of the target function with respect to transformations of its inputs. Consider a learning task involving an instance space X and target function f . Up to now we have assumed that each training example consists of a pair $\langle x_i, f(x_i) \rangle$ that describes some instance x_i and its training value $f(x_i)$. The TANGENTPROP algorithm assumes various training derivatives of the target function are also provided. For example, if each instance x_i is described by a single real value, then each training example may be of the form $\langle x_i, f(x_i), \frac{\partial f(x)}{\partial x}|_{x_i} \rangle$. Here $\frac{\partial f(x)}{\partial x}|_{x_i}$ denotes the derivative of the target function f with respect to x , evaluated at the point $x = x_i$.

To develop an intuition for the benefits of providing training derivatives as well as training values during learning, consider the simple learning task depicted in Figure 12.5. The task is to learn the target function f shown in the leftmost plot of the figure, based on the three training examples shown: $\langle x_1, f(x_1) \rangle$, $\langle x_2, f(x_2) \rangle$, and $\langle x_3, f(x_3) \rangle$. Given these three training examples, the BACKPROPAGATION algorithm can be expected to hypothesize a smooth function, such as the function g depicted in the middle plot of the figure. The rightmost plot shows the effect of

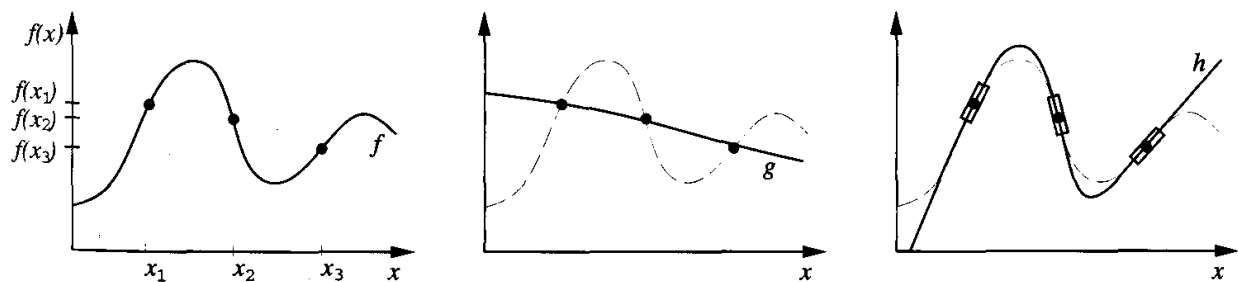


FIGURE 12.5

Fitting values and derivatives with TANGENTPROP. Let f be the target function for which three examples $\langle x_1, f(x_1) \rangle$, $\langle x_2, f(x_2) \rangle$, and $\langle x_3, f(x_3) \rangle$ are known. Based on these points the learner might generate the hypothesis g . If the derivatives are also known, the learner can generalize more accurately h .

providing training derivatives, or slopes, as additional information for each training example (e.g., $(x_1, f(x_1), \frac{\partial f(x)}{\partial x}|_{x_1})$). By fitting both the training values $f(x_i)$ and these training derivatives $\frac{\partial f(x)}{\partial x}|_{x_i}$, the learner has a better chance to correctly generalize from the sparse training data. To summarize, the impact of including the training derivatives is to override the usual syntactic inductive bias of BACKPROPAGATION that favors a smooth interpolation between points, replacing it by explicit input information about required derivatives. The resulting hypothesis h shown in the rightmost plot of the figure provides a much more accurate estimate of the true target function f .

In the above example, we considered only simple kinds of derivatives of the target function. In fact, TANGENTPROP can accept training derivatives with respect to various transformations of the input x . Consider, for example, the task of learning to recognize handwritten characters. In particular, assume the input x corresponds to an image containing a single handwritten character, and the task is to correctly classify the character. In this task, we might be interested in informing the learner that “the target function is invariant to small rotations of the character within the image.” In order to express this prior knowledge to the learner, we first define a transformation $s(\alpha, x)$, which rotates the image x by α degrees. Now we can express our assertion about rotational invariance by stating that for each training instance x_i , the derivative of the target function with respect to this transformation is zero (i.e., that rotating the input image does not alter the value of the target function). In other words, we can assert the following training derivative for every training instance x_i

$$\frac{\partial f(s(\alpha, x_i))}{\partial \alpha} = 0$$

where f is the target function and $s(\alpha, x_i)$ is the image resulting from applying the transformation s to the image x_i .

How are such training derivatives used by TANGENTPROP to constrain the weights of the neural network? In TANGENTPROP these training derivatives are incorporated into the error function that is minimized by gradient descent. Recall from Chapter 4 that the BACKPROPAGATION algorithm performs gradient descent to attempt to minimize the sum of squared errors

$$E = \sum_i (f(x_i) - \hat{f}(x_i))^2$$

where x_i denotes the i th training instance, f denotes the true target function, and $\hat{f}$ denotes the function represented by the learned neural network.

In TANGENTPROP an additional term is added to the error function to penalize discrepancies between the training derivatives and the actual derivatives of the learned neural network function $\hat{f}$. In general, TANGENTPROP accepts multiple transformations (e.g., we might wish to assert both rotational invariance and translational invariance of the character identity). Each transformation must be of the form $s_j(\alpha, x)$ where α is a continuous parameter, where s_j is differentiable, and where $s_j(0, x) = x$ (e.g., for rotation of zero degrees the transformation is the identity function). For each such transformation, $s_j(\alpha, x)$, TANGENT-

PROP considers the squared error between the specified training derivative and the actual derivative of the learned neural network. The modified error function is

$$E = \sum_i \left[(f(x_i) - \hat{f}(x_i))^2 + \mu \sum_j \left(\frac{\partial f(s_j(\alpha, x_i))}{\partial \alpha} - \frac{\partial \hat{f}(s_j(\alpha, x_i))}{\partial \alpha} \right)^2 \right]_{\alpha=0} \quad (12.1)$$

where μ is a constant provided by the user to determine the relative importance of fitting training values versus fitting training derivatives. Notice the first term in this definition of E is the original squared error of the network versus training values, and the second term is the squared error in the network versus training derivatives.

Simard et al. (1992) give the gradient descent rule for minimizing this extended error function E . It can be derived in a fashion analogous to the derivation given in Chapter 4 for the simpler BACKPROPAGATION rule.

12.4.2 An Illustrative Example

Simard et al. (1992) present results comparing the generalization accuracy of TANGENTPROP and purely inductive BACKPROPAGATION for the problem of recognizing handwritten characters. More specifically, the task in this case is to label images containing a single digit between 0 and 9. In one experiment, both TANGENTPROP and BACKPROPAGATION were trained using training sets of varying size, then evaluated based on their performance over a separate test set of 160 examples. The prior knowledge given to TANGENTPROP was the fact that the classification of the digit is invariant of vertical and horizontal translation of the image (i.e., that the derivative of the target function was 0 with respect to these transformations). The results, shown in Table 12.4, demonstrate the ability of TANGENTPROP using this prior knowledge to generalize more accurately than purely inductive BACKPROPAGATION.

Training set size	Percent error on test set	
	TANGENTPROP	BACKPROPAGATION
10	34	48
20	17	33
40	7	18
80	4	10
160	0	3
320	0	0

TABLE 12.4

Generalization accuracy for TANGENTPROP and BACKPROPAGATION, for handwritten digit recognition. TANGENTPROP generalizes more accurately due to its prior knowledge that the identity of the digit is invariant of translation. These results are from Simard et al. (1992).

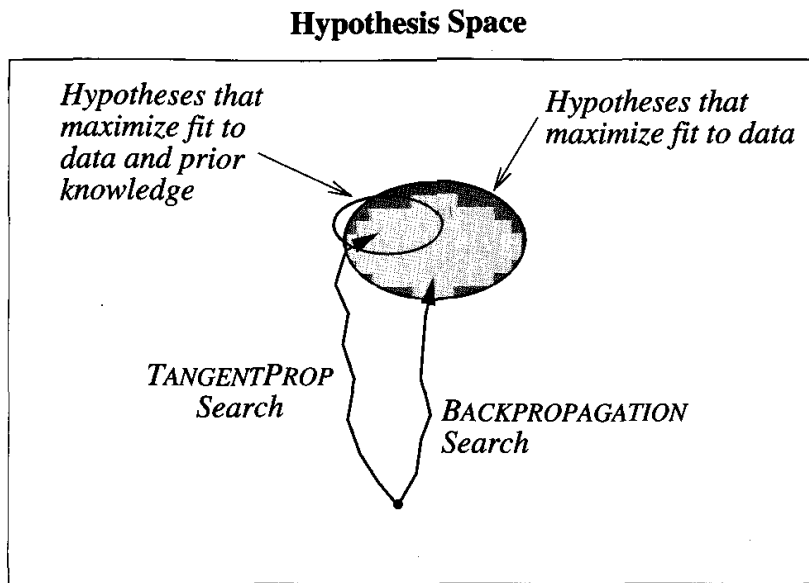
12.4.3 Remarks

To summarize, TANGENTPROP uses prior knowledge in the form of desired derivatives of the target function with respect to transformations of its inputs. It combines this prior knowledge with observed training data, by minimizing an objective function that measures both the network's error with respect to the training example values (fitting the data) and its error with respect to the desired derivatives (fitting the prior knowledge). The value of μ determines the degree to which the network will fit one or the other of these two components in the total error. The behavior of the algorithm is sensitive to μ , which must be chosen by the designer.

Although TANGENTPROP succeeds in combining prior knowledge with training data to guide learning of neural networks, it is not robust to errors in the prior knowledge. Consider what will happen when prior knowledge is incorrect, that is, when the training derivatives input to the learner do not correctly reflect the derivatives of the true target function. In this case the algorithm will attempt to fit incorrect derivatives. It may therefore generalize less accurately than if it ignored this prior knowledge altogether and used the purely inductive BACKPROPAGATION algorithm. If we knew in advance the degree of error in the training derivatives, we might use this information to select the constant μ that determines the relative importance of fitting training values and fitting training derivatives. However, this information is unlikely to be known in advance. In the next section we discuss the EBNN algorithm, which automatically selects values for μ on an example-by-example basis in order to address the possibility of incorrect prior knowledge.

It is interesting to compare the search through hypothesis space (weight space) performed by TANGENTPROP, KBANN, and BACKPROPAGATION. TANGENTPROP incorporates prior knowledge to influence the hypothesis search by altering the objective function to be minimized by gradient descent. This corresponds to altering the goal of the hypothesis space search, as illustrated in Figure 12.6. Like BACKPROPAGATION (but unlike KBANN), TANGENTPROP begins the search with an initial network of small random weights. However, the gradient descent training rule produces different weight updates than BACKPROPAGATION, resulting in a different final hypothesis. As shown in the figure, the set of hypotheses that minimizes the TANGENTPROP objective may differ from the set that minimizes the BACKPROPAGATION objective. Importantly, if the training examples and prior knowledge are both correct, and the target function can be accurately represented by the ANN, then the set of weight vectors that satisfy the TANGENTPROP objective will be a subset of those satisfying the weaker BACKPROPAGATION objective. The difference between these two sets of final hypotheses is the set of incorrect hypotheses that will be considered by BACKPROPAGATION, but ruled out by TANGENTPROP due to its prior knowledge.

Note one alternative to fitting the training derivatives of the target function is to simply synthesize additional training examples near the observed training examples, using the known training derivatives to estimate training values for these nearby instances. For example, one could take a training image in the above character recognition task, translate it a small amount, and assert that the trans-

**FIGURE 12.6**

Hypothesis space search in TANGENTPROP. TANGENTPROP initializes the network to small random weights, just as in BACKPROPAGATION. However, it uses a different error function to drive the gradient descent search. The error used by TANGENTPROP includes both the error in predicting training *values* and in predicting the training *derivatives* provided as prior knowledge.

lated image belonged to the same class as the original example. We might expect that fitting these synthesized examples using BACKPROPAGATION would produce results similar to fitting the original training examples and derivatives using TANGENTPROP. Simard et al. (1992) report experiments showing similar generalization error in the two cases, but report that TANGENTPROP converges considerably more efficiently. It is interesting to note that the ALVINN system, which learns to steer an autonomous vehicle (see Chapter 4), uses a very similar approach to synthesize additional training examples. It uses prior knowledge of how the desired steering direction changes with horizontal translation of the camera image to create multiple synthetic training examples to augment each observed training example.

12.4.4 The EBNN Algorithm

The EBNN (Explanation-Based Neural Network learning) algorithm (Mitchell and Thrun 1993a; Thrun 1996) builds on the TANGENTPROP algorithm in two significant ways. First, instead of relying on the user to provide training derivatives, EBNN computes training derivatives itself for each observed training example. These training derivatives are calculated by explaining each training example in terms of a given domain theory, then extracting training derivatives from this explanation. Second, EBNN addresses the issue of how to weight the relative importance of the inductive and analytical components of learning (i.e., how to select the parameter μ in Equation [12.1]). The value of μ is chosen independently for each training example, based on a heuristic that considers how accurately the domain theory predicts the training value for this particular example. Thus, the analytical component of learning is emphasized for those training examples that are correctly

explained by the domain theory and de-emphasized for training examples that are poorly explained.

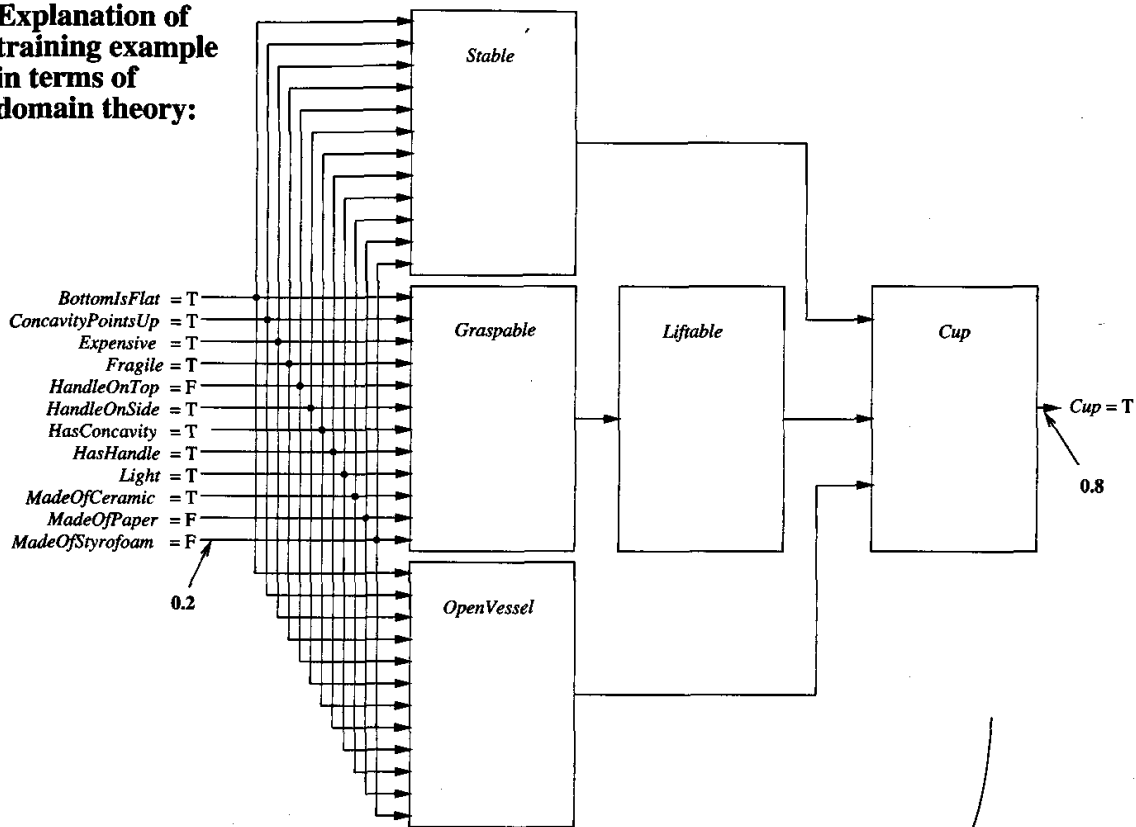
The inputs to EBNN include (1) a set of training examples of the form $\langle x_i, f(x_i) \rangle$ with no training derivatives provided, and (2) a domain theory analogous to that used in explanation-based learning (Chapter 11) and in KBANN, but represented by a set of previously trained neural networks rather than a set of Horn clauses. The output of EBNN is a new neural network that approximates the target function f . This learned network is trained to fit both the training examples $\langle x_i, f(x_i) \rangle$ and training derivatives of f extracted from the domain theory. Fitting the training examples $\langle x_i, f(x_i) \rangle$ constitutes the inductive component of learning, whereas fitting the training derivatives extracted from the domain theory provides the analytical component.

To illustrate the type of domain theory used by EBNN, consider Figure 12.7. The top portion of this figure depicts an EBNN domain theory for the target function *Cup*, with each rectangular block representing a distinct neural network in the domain theory. Notice in this example there is one network for each of the Horn clauses in the symbolic domain theory of Table 12.3. For example, the network labeled *Graspable* takes as input the description of an instance and produces as output a value indicating whether the object is graspable (EBNN typically represents true propositions by the value 0.8 and false propositions by the value 0.2). This network is analogous to the Horn clause for *Graspable* given in Table 12.3. Some networks take the outputs of other networks as their inputs (e.g., the right-most network labeled *Cup* takes its inputs from the outputs of the *Stable*, *Liftable*, and *OpenVessel* networks). Thus, the networks that make up the domain theory can be chained together to infer the target function value for the input instance, just as Horn clauses might be chained together for this purpose. In general, these domain theory networks may be provided to the learner by some external source, or they may be the result of previous learning by the same system. EBNN makes use of these domain theory networks to learn the new target function. It does not alter the domain theory networks during this process.

The goal of EBNN is to learn a new neural network to describe the target function. We will refer to this new network as the *target network*. In the example of Figure 12.7, the target network Cup_{target} shown at the bottom of the figure takes as input the description of an arbitrary instance and outputs a value indicating whether the object is a *Cup*.

EBNN learns the target network by invoking the TANGENTPROP algorithm described in the previous section. Recall that TANGENTPROP trains a network to fit both training values and training derivatives. EBNN passes along to TANGENTPROP the training values $\langle x_i, f(x_i) \rangle$ that it receives as input. In addition, EBNN provides TANGENTPROP with derivatives that it calculates from the domain theory. To see how EBNN calculates these training derivatives, consider again Figure 12.7. The top portion of this figure shows the domain theory prediction of the target function value for a particular training instance, x_i . EBNN calculates the derivative of this prediction with respect to each feature of the input instance. For the example in the figure, the instance x_i is described by features such as *MadeOfStyrofoam* = 0.2

Explanation of training example in terms of domain theory:



Target network:

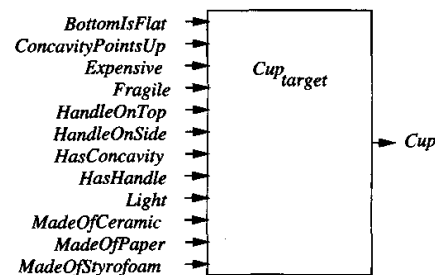


FIGURE 12.7

Explanation of a training example in EBNN. The explanation consists of a prediction of the target function value by the domain theory networks (top). Training derivatives are extracted from this explanation in order to train the separate target network (bottom). Each rectangular block represents a distinct multilayer neural network.

(i.e., *False*), and the domain theory prediction is that $Cup = 0.8$ (i.e., *True*). EBNN calculates the partial derivative of this prediction with respect to each instance feature, yielding the set of derivatives

$$\left[\frac{\partial Cup}{\partial BottomIsFlat}, \frac{\partial Cup}{\partial ConcavityPointsUp}, \dots, \frac{\partial Cup}{\partial MadeOfStyrofoam} \right]_{x=x_i}$$

This set of derivatives is the gradient of the domain theory prediction function with respect to the input instance. The subscript refers to the fact that these derivatives

hold when $x = x_i$. In the more general case where the target function has multiple output units, the gradient is computed for each of these outputs. This matrix of gradients is called the Jacobian of the target function.

To see the importance of these training derivatives in helping to learn the target network, consider the derivative $\frac{\partial Cup}{\partial Expensive}$. If the domain theory encodes the knowledge that the feature *Expensive* is irrelevant to the target function *Cup*, then the derivative $\frac{\partial Cup}{\partial Expensive}$ extracted from the explanation will have the value zero. A derivative of zero corresponds to the assertion that a change in the feature *Expensive* will have no impact on the predicted value of *Cup*. On the other hand, a large positive or negative derivative corresponds to the assertion that the feature is highly relevant to determining the target value. Thus, the derivatives extracted from the domain theory explanation provide important information for distinguishing relevant from irrelevant features. When these extracted derivatives are provided as training derivatives to TANGENTPROP for learning the target network Cup_{target} , they provide a useful bias for guiding generalization. The usual syntactic inductive bias of neural network learning is replaced in this case by the bias exerted by the derivatives obtained from the domain theory.

Above we described how the domain theory prediction can be used to generate a set of training derivatives. To be more precise, the full EBNN algorithm is as follows. Given the training examples and domain theory, EBNN first creates a new, fully connected feedforward network to represent the target function. This target network is initialized with small random weights, just as in BACKPROPAGATION. Next, for each training example $\langle x_i, f(x_i) \rangle$ EBNN determines the corresponding training derivatives in a two-step process. First, it uses the domain theory to predict the value of the target function for instance x_i . Let $A(x_i)$ denote this domain theory prediction for instance x_i . In other words, $A(x_i)$ is the function defined by the composition of the domain theory networks forming the explanation for x_i . Second, the weights and activations of the domain theory networks are analyzed to extract the derivatives of $A(x_i)$ with respect to each of the components of x_i (i.e., the Jacobian of $A(x)$ evaluated at $x = x_i$). Extracting these derivatives follows a process very similar to calculating the δ terms in the BACKPROPAGATION algorithm (see Exercise 12.5). Finally, EBNN uses a minor variant of the TANGENTPROP algorithm to train the target network to fit the following error function

$$E = \sum_i \left[(f(x_i) - \hat{f}(x_i))^2 + \mu_i \sum_j \left(\frac{\partial A(x)}{\partial x^j} - \frac{\partial \hat{f}(x)}{\partial x^j} \right)^2_{(x=x_i)} \right] \quad (12.2)$$

where

$$\mu_i \equiv 1 - \frac{|A(x_i) - f(x_i)|}{c} \quad (12.3)$$

Here x_i denotes the i th training instance and $A(x)$ denotes the domain theory prediction for input x . The superscript notation x^j denotes the j th component of the vector x (i.e., the j th input node of the neural network). The coefficient c is a normalizing constant whose value is chosen to assure that for all i , $0 \leq \mu_i \leq 1$.

Although the notation here appears a bit tedious, the idea is simple. The error given by Equation (12.2) has the same general form as the error function in Equation (12.1) minimized by TANGENTPROP. The leftmost term measures the usual sum of squared errors between the training value $f(x_i)$ and the value predicted by the target network $\hat{f}(x_i)$. The rightmost term measures the squared error between the training derivatives $\frac{\partial A(x)}{\partial x^j}$ extracted from the domain theory and the actual derivatives of the target network $\frac{\partial \hat{f}(x)}{\partial x^j}$. Thus, the leftmost term contributes the inductive constraint that the hypothesis must fit the observed training data, whereas the rightmost term contributes the analytical constraint that it must fit the training derivatives extracted from the domain theory. Notice the derivative $\frac{\partial \hat{f}(x)}{\partial x^j}$ in Equation (12.2) is just a special case of the expression $\frac{\partial \hat{f}(s_j(\alpha, x_i))}{\partial \alpha}$ of Equation (12.1), for which $s_j(\alpha, x_i)$ is the transformation that replaces x_i^j by $x_i^j + \alpha$. The precise weight-training rule used by EBNN is described by Thrun (1996).

The relative importance of the inductive and analytical learning components is determined in EBNN by the constant μ_i , defined in Equation (12.3). The value of μ_i is determined by the discrepancy between the domain theory prediction $A(x_i)$ and the training value $f(x_i)$. The analytical component of learning is thus weighted more heavily for training examples that are correctly predicted by the domain theory and is suppressed for examples that are not correctly predicted. This weighting heuristic assumes that the training *derivatives* extracted from the domain theory are more likely to be correct in cases where the training *value* is correctly predicted by the domain theory. Although one can construct situations in which this heuristic fails, in practice it has been found effective in several domains (e.g., see Mitchell and Thrun [1993a]; Thrun [1996]).

12.4.5 Remarks

To summarize, the EBNN algorithm uses a domain theory expressed as a set of previously learned neural networks, together with a set of training examples, to train its output hypothesis (the target network). For each training example EBNN uses its domain theory to explain the example, then extracts training derivatives from this explanation. For each attribute of the instance, a training derivative is computed that describes how the target function value is influenced by a small change to this attribute value, according to the domain theory. These training derivatives are provided to a variant of TANGENTPROP, which fits the target network to these derivatives and to the training example values. Fitting the derivatives constrains the learned network to fit dependencies given by the domain theory, while fitting the training values constrains it to fit the observed data itself. The weight μ_i placed on fitting the derivatives is determined independently for each training example, based on how accurately the domain theory predicts the training value for this example.

EBNN has been shown to be an effective method for learning from approximate domain theories in several domains. Thrun (1996) describes its application to a variant of the *Cup* learning task discussed above and reports that

EBNN generalizes more accurately than standard BACKPROPAGATION, especially when training data is scarce. For example, after 30 training examples, EBNN achieved a root-mean-squared error of 5.5 on a separate set of test data, compared to an error of 12.0 for BACKPROPAGATION. Mitchell and Thrun (1993a) describe applying EBNN to learning to control a simulated mobile robot, in which the domain theory consists of neural networks that predict the effects of various robot actions on the world state. Again, EBNN using an approximate, previously learned domain theory, outperformed BACKPROPAGATION. Here BACKPROPAGATION required approximately 90 training episodes to reach the level of performance achieved by EBNN after 25 training episodes. O'Sullivan et al. (1997) and Thrun (1996) describe several other applications of EBNN to real-world robot perception and control tasks, in which the domain theory consists of networks that predict the effect of actions for an indoor mobile robot using sonar, vision, and laser range sensors.

EBNN bears an interesting relation to other explanation-based learning methods, such as PROLOG-EBG described in Chapter 11. Recall from that chapter that PROLOG-EBG also constructs explanations (predictions of example target values) based on a domain theory. In PROLOG-EBG the explanation is constructed from a domain theory consisting of Horn clauses, and the target hypothesis is refined by calculating the weakest conditions under which this explanation holds. Relevant dependencies in the explanation are thus captured in the learned Horn clause hypothesis. EBNN constructs an analogous explanation, but it is based on a domain theory consisting of neural networks rather than Horn clauses. As in PROLOG-EBG, relevant dependencies are then extracted from the explanation and used to refine the target hypothesis. In the case of EBNN, these dependencies take the form of derivatives because derivatives are the natural way to represent dependencies in continuous functions such as neural networks. In contrast, the natural way to represent dependencies in symbolic explanations or logical proofs is to describe the set of examples to which the proof applies.

There are several differences in capabilities between EBNN and the symbolic explanation-based methods of Chapter 11. The main difference is that EBNN accommodates imperfect domain theories, whereas PROLOG-EBG does not. This difference follows from the fact that EBNN is built on the inductive mechanism of fitting the observed training values and uses the domain theory only as an additional constraint on the learned hypothesis. A second important difference follows from the fact that PROLOG-EBG learns a growing set of Horn clauses, whereas EBNN learns a fixed-size neural network. As discussed in Chapter 11, one difficulty in learning sets of Horn clauses is that the cost of classifying a new instance grows as learning proceeds and new Horn clauses are added. This problem is avoided in EBNN because the fixed-size target network requires constant time to classify new instances. However, the fixed-size neural network suffers the corresponding disadvantage that it may be unable to represent sufficiently complex functions, whereas a growing set of Horn clauses can represent increasingly complex functions. Mitchell and Thrun (1993b) provide a more detailed discussion of the relationship between EBNN and symbolic explanation-based learning methods.

12.5 USING PRIOR KNOWLEDGE TO AUGMENT SEARCH OPERATORS

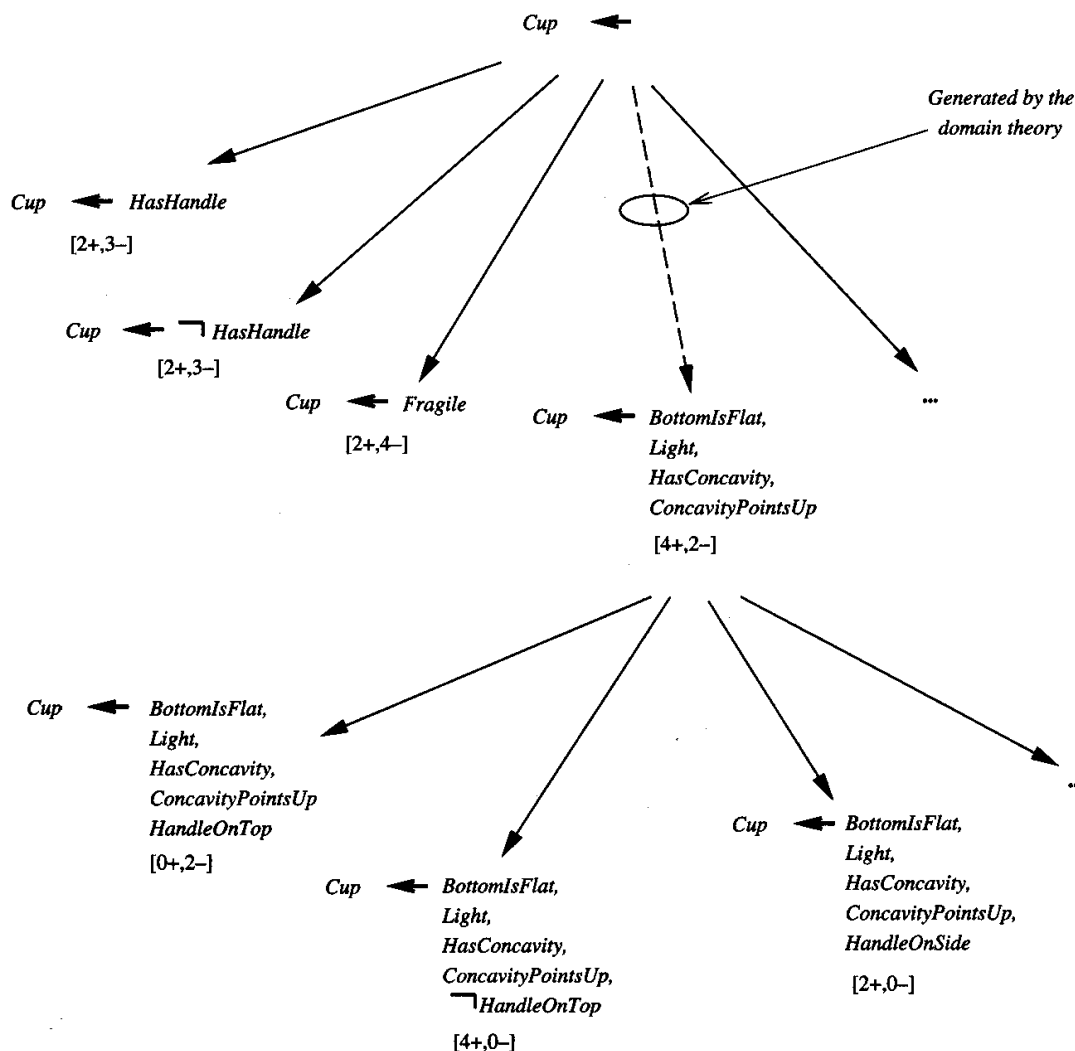
The two previous sections examined two different roles for prior knowledge in learning: initializing the learner's hypothesis and altering the objective function that guides search through the hypothesis space. In this section we consider a third way of using prior knowledge to alter the hypothesis space search: using it to alter the set of operators that define legal steps in the search through the hypothesis space. This approach is followed by systems such as FOCL (Pazzani et al. 1991; Pazzani and Kibler 1992) and ML-SMART (Bergadano and Giordana 1990). Here we use FOCL to illustrate the approach.

12.5.1 The FOCL Algorithm

FOCL is an extension of the purely inductive FOIL system described in Chapter 10. Both FOIL and FOCL learn a set of first-order Horn clauses to cover the observed training examples. Both systems employ a sequential covering algorithm that learns a single Horn clause, removes the positive examples covered by this new Horn clause, and then iterates this procedure over the remaining training examples. In both systems, each new Horn clause is created by performing a general-to-specific search, beginning with the most general possible Horn clause (i.e., a clause containing no preconditions). Several candidate specializations of the current clause are then generated, and the specialization with greatest information gain relative to the training examples is chosen. This process is iterated, generating further candidate specializations and selecting the best, until a Horn clause with satisfactory performance is obtained.

The difference between FOIL and FOCL lies in the way in which candidate specializations are generated during the general-to-specific search for a single Horn clause. As described in Chapter 10, FOIL generates each candidate specialization by adding a single new literal to the clause preconditions. FOCL uses this same method for producing candidate specializations, but also generates additional specializations based on the domain theory. The *solid* edges in the search tree of Figure 12.8 show the general-to-specific search steps considered in a typical search by FOIL. The *dashed* edge in the search tree of Figure 12.8 denotes an additional candidate specialization that is considered by FOCL and based on the domain theory.

Although FOCL and FOIL both learn first-order Horn clauses, we illustrate their operation here using the simpler domain of propositional (variable-free) Horn clauses. In particular, consider again the *Cup* target concept, training examples, and domain theory from Figure 12.3. To describe the operation of FOCL, we must first draw a distinction between two kinds of literals that appear in the domain theory and hypothesis representation. We will say a literal is *operational* if it is allowed to be used in describing an output hypothesis. For example, in the *Cup* example of Figure 12.3 we allow output hypotheses to refer only to the 12 attributes that describe the training examples (e.g., *HasHandle*, *HandleOnTop*). Literals based on these 12 attributes are thus considered operational. In contrast, literals that occur only as intermediate features in the domain theory, but not as

**FIGURE 12.8**

Hypothesis space search in FOCL. To learn a single rule, FOCL searches from general to increasingly specific hypotheses. Two kinds of operators generate specializations of the current hypothesis. One kind adds a single new literal (solid lines in the figure). A second kind of operator specializes the rule by adding a set of literals that constitute logically sufficient conditions for the target concept, according to the domain theory (dashed lines in the figure). FOCL selects among all these candidate specializations, based on their performance over the data. Therefore, imperfect domain theories will impact the hypothesis only if the evidence supports the theory. This example is based on the same training data and domain theory as the earlier KBANN example.

primitive attributes of the instances, are considered nonoperational. An example of a nonoperational attribute in this case is the attribute *Stable*.

At each point in its general-to-specific search, FOCL expands its current hypothesis h using the following two operators:

1. For each *operational* literal that is not part of h , create a specialization of h by adding this single literal to the preconditions. This is also the method used by FOIL to generate candidate successors. The solid arrows in Figure 12.8 denote this type of specialization.

2. Create an operational, logically sufficient condition for the target concept according to the domain theory. Add this set of literals to the current preconditions of h . Finally, prune the preconditions of h by removing any literals that are unnecessary according to the training data. The dashed arrow in Figure 12.8 denotes this type of specialization.

The detailed procedure for the second operator above is as follows. FOCL first selects one of the domain theory clauses whose head (postcondition) matches the target concept. If there are several such clauses, it selects the clause whose body (preconditions) have the highest information gain relative to the training examples of the target concept. For example, in the domain theory and training data of Figure 12.3, there is only one such clause:

Cup $\leftarrow$ *Stable, Lifiable, OpenVessel*

The preconditions of the selected clause form a logically sufficient condition for the target concept. Each nonoperational literal in these sufficient conditions is now replaced, again using the domain theory and substituting clause preconditions for clause postconditions. For example, the domain theory clause *Stable* $\leftarrow$ *BottomIsFlat* is used to substitute the operational *BottomIsFlat* for the unoperational *Stable*. This process of “unfolding” the domain theory continues until the sufficient conditions have been restated in terms of operational literals. If there are several alternative domain theory clauses that produce different results, then the one with the greatest information gain is greedily selected at each step of the unfolding process. The reader can verify that the final operational sufficient condition given the data and domain theory in the current example is

BottomIsFlat, HasHandle, Light, HasConcavity, ConcavityPointsUp

As a final step in generating the candidate specialization, this sufficient condition is pruned. For each literal in the expression, the literal is removed unless its removal reduces classification accuracy over the training examples. This step is included to recover from overspecialization in case the imperfect domain theory includes irrelevant literals. In our current example, the above set of literals matches two positive and two negative examples. Pruning (removing) the literal *HasHandle* results in improved performance. The final pruned, operational, sufficient conditions are, therefore,

BottomIsFlat, Light, HasConcavity, ConcavityPointsUp

This set of literals is now added to the preconditions of the current hypothesis. Note this hypothesis is the result of the search step shown by the dashed arrow in Figure 12.8.

Once candidate specializations of the current hypothesis have been generated, using both of the two operations above, the candidate with highest information gain is selected. In the example shown in Figure 12.8 the candidate chosen at the first level in the search tree is the one generated by the domain theory. The search then proceeds by considering further specializations of the theory-suggested

preconditions, thereby allowing the inductive component of learning to refine the preconditions derived from the domain theory. In this example, the domain theory affects the search only at the first search level. However, this will not always be the case. Should the empirical support be stronger for some other candidate at the first level, theory-suggested literals may still be added at subsequent steps in the search. To summarize, FOCL learns Horn clauses of the form

$$c \leftarrow o_i \wedge o_b \wedge o_f$$

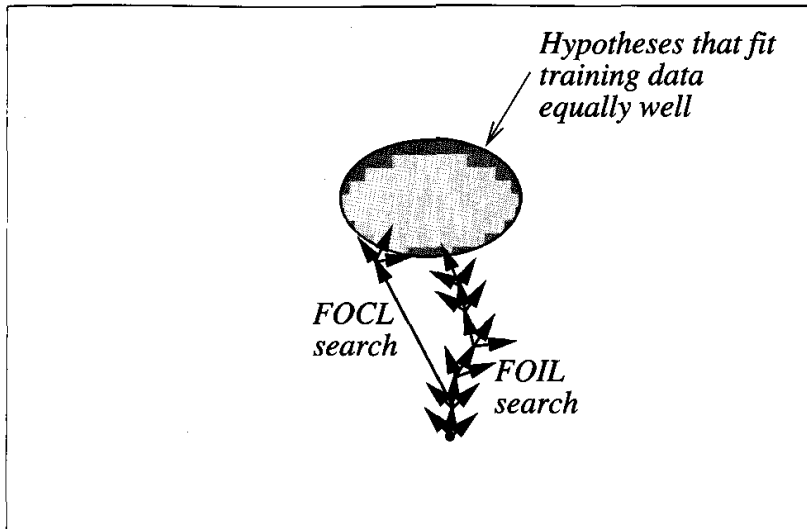
where c is the target concept, o_i is an initial conjunction of operational literals added one at a time by the first syntactic operator, o_b is a conjunction of operational literals added in a single step based on the domain theory, and o_f is a final conjunction of operational literals added one at a time by the first syntactic operator. Any of these three sets of literals may be empty.

The above discussion illustrates the use of a propositional domain theory to create candidate specializations of the hypothesis during the general-to-specific search for a single Horn clause. The algorithm is easily extended to first-order representations (i.e., representations including variables). Chapter 10 discusses in detail the algorithm used by FOIL to generate first-order Horn clauses, including the extension of the first of the two search operators described above to first-order representations. To extend the second operator to accommodate first-order domain theories, variable substitutions must be considered when unfolding the domain theory. This can be accomplished using a procedure related to the regression procedure described in Table 11.3.

12.5.2 Remarks

FOCL uses the domain theory to increase the number of candidate specializations considered at each step of the search for a single Horn clause. Figure 12.9 compares the hypothesis space search performed by FOCL to that performed by the purely inductive FOIL algorithm on which it is based. FOCL's theory-suggested specializations correspond to "macro" steps in FOIL's search, in which several literals are added in a single step. This process can be viewed as promoting a hypothesis that might be considered later in the search to one that will be considered immediately. If the domain theory is correct, the training data will bear out the superiority of this candidate over the others and it will be selected. If the domain theory is incorrect, the empirical evaluation of all the candidates should direct the search down an alternative path.

To summarize, FOCL uses both a syntactic generation of candidate specializations and a domain theory driven generation of candidate specializations at each step in the search. The algorithm chooses among these candidates based solely on their empirical support over the training data. Thus, the domain theory is used in a fashion that biases the learner, but leaves final search choices to be made based on performance over the training data. The bias introduced by the domain theory is a preference in favor of Horn clauses most similar to operational, logically sufficient conditions entailed by the domain theory. This bias is combined with

Hypothesis Space**FIGURE 12.9**

Hypothesis space search in FOCL. FOCL augments the set of search operators used by FOIL. Whereas FOIL considers adding a single new literal at each step, FOIL also considers adding multiple literals derived from the domain theory.

the bias of the purely inductive FOIL program, which is a preference for shorter hypotheses.

FOCL has been shown to generalize more accurately than the purely inductive FOIL algorithm in a number of application domains in which an imperfect domain theory is available. For example, Pazzani and Kibler (1992) explore learning the concept “legal chessboard positions.” Given 60 training examples describing 30 legal and 30 illegal endgame board positions, FOIL achieved an accuracy of 86% over an independent set of test examples. FOCL was given the same 60 training examples, along with an approximate domain theory with an accuracy of 76%. FOCL produced a hypothesis with generalization accuracy of 94%—less than half the error rate of FOIL. Similar results have been obtained in other domains. For example, given 500 training examples of telephone network problems and their diagnoses from the telephone company NYNEX, FOIL achieved an accuracy of 90%, whereas FOCL reached an accuracy of 98% when given the same training data along with a 95% accurate domain theory.

12.6 STATE OF THE ART

The methods presented in this chapter are only a sample of the possible approaches to combining analytical and inductive learning. While each of these methods has been demonstrated to outperform purely inductive learning methods in selected domains, none of these has been thoroughly tested or proven across a large variety of problem domains. The topic of combining inductive and analytical learning remains a very active research area.

12.7 SUMMARY AND FURTHER READING

The main points of this chapter include:

- Approximate prior knowledge, or domain theories, are available in many practical learning problems. Purely inductive methods such as decision tree induction and neural network BACKPROPAGATION fail to utilize such domain theories, and therefore perform poorly when data is scarce. Purely analytical learning methods such as PROLOG-EBG utilize such domain theories, but produce incorrect hypotheses when given imperfect prior knowledge. Methods that blend inductive and analytical learning can gain the benefits of both approaches: reduced sample complexity and the ability to overrule incorrect prior knowledge.
- One way to view algorithms for combining inductive and analytical learning is to consider how the domain theory affects the hypothesis space search. In this chapter we examined methods that use imperfect domain theories to (1) create the initial hypothesis in the search, (2) expand the set of search operators that generate revisions to the current hypothesis, and (3) alter the objective of the search.
- A system that uses the domain theory to initialize the hypothesis is KBANN. This algorithm uses a domain theory encoded as propositional rules to analytically construct an artificial neural network that is equivalent to the domain theory. This network is then inductively refined using the BACKPROPAGATION algorithm, to improve its performance over the training data. The result is a network biased by the original domain theory, whose weights are refined inductively based on the training data.
- TANGENTPROP uses prior knowledge represented by desired derivatives of the target function. In some domains, such as image processing, this is a natural way to express prior knowledge. TANGENTPROP incorporates this knowledge by altering the objective function minimized by gradient descent search through the space of possible hypotheses.
- EBNN uses the domain theory to alter the objective in searching the hypothesis space of possible weights for an artificial neural network. It uses a domain theory consisting of previously learned neural networks to perform a neural network analog to symbolic explanation-based learning. As in symbolic explanation-based learning, the domain theory is used to explain individual examples, yielding information about the relevance of different example features. With this neural network representation, however, information about relevance is expressed in the form of derivatives of the target function value with respect to instance features. The network hypothesis is trained using a variant of the TANGENTPROP algorithm, in which the error to be minimized includes both the error in network output values and the error in network derivatives obtained from explanations.
- FOCL uses the domain theory to expand the set of candidates considered at each step in the search. It uses an approximate domain theory represented

by first order Horn clauses to learn a set of Horn clauses that approximate the target function. FOCL employs a sequential covering algorithm, learning each Horn clause by a general-to-specific search. The domain theory is used to augment the set of next more specific candidate hypotheses considered at each step of this search. Candidate hypotheses are then evaluated based on their performance over the training data. In this way, FOCL combines the greedy, general-to-specific inductive search strategy of FOIL with the rule-chaining, analytical reasoning of analytical methods.

- The question of how to best blend prior knowledge with new observations remains one of the key open questions in machine learning.

There are many more examples of algorithms that attempt to combine inductive and analytical learning. For example, methods for learning Bayesian belief networks discussed in Chapter 6 provide one alternative to the approaches discussed here. The references at the end of this chapter provide additional examples and sources for further reading.

EXERCISES

- 12.1. Consider learning the target concept *GoodCreditRisk* defined over instances described by the four attributes *HasStudentLoan*, *HasSavingsAccount*, *IsStudent*, *OwnsCar*. Give the initial network created by KBANN for the following domain theory, including all network connections and weights.

$$\begin{aligned} \text{GoodCreditRisk} &\leftarrow \text{Employed}, \text{LowDebt} \\ \text{Employed} &\leftarrow \neg \text{IsStudent} \\ \text{LowDebt} &\leftarrow \neg \text{HasStudentLoan}, \text{HasSavingsAccount} \end{aligned}$$

- 12.2. KBANN converts a set of propositional Horn clauses into an initial neural network. Consider the class of *n-of-m clauses*, which are Horn clauses containing *m* literals in the preconditions (antecedents), and an associated parameter *n* where $n \leq m$. The preconditions of an *n-of-m* Horn clause are considered to be satisfied if at least *n* of its *m* preconditions are satisfied. For example, the clause

$$\text{Student} \leftarrow \text{LivesInDorm}, \text{Young}, \text{Studies}; \quad n = 2$$

asserts that one is a *Student* if at least two of these three preconditions are satisfied.

Give an algorithm similar to that used by KBANN, that accepts a set of propositional *n-of-m* clauses and constructs a neural network consistent with the domain theory.

- 12.3. Consider extending KBANN to accept a domain theory consisting of first-order rather than propositional Horn clauses (i.e., Horn clauses containing variables, as in Chapter 10). Either give an algorithm for constructing a neural network equivalent to a set of Horn clauses, or discuss the difficulties that prevent this.
- 12.4. This exercise asks you to derive a gradient descent rule analogous to that used by TANGENTPROP. Consider the instance space *X* consisting of the real numbers, and consider the hypothesis space *H* consisting of quadratic functions of *x*. That is,

each hypothesis $h(x)$ is of the form

$$h(x) = w_0 + w_1x + w_2x^2$$

- (a) Derive a gradient descent rule that minimizes the same criterion as BACKPROPAGATION; that is, the sum of squared errors between the hypothesis and target values of the training data.
 - (b) Derive a second gradient descent rule that minimizes the same criterion as TANGENTPROP. Consider only the single transformation $s(\alpha, x) = x + \alpha$.
- 12.5.** EBNN extracts training derivatives from explanations by examining the weights and activations of the neural networks that make up the explanation. Consider the simple example in which the explanation is formed by a single sigmoid unit with n inputs. Derive a procedure for extracting the derivative $\frac{\partial \hat{f}(x)}{\partial x^j} \big|_{x=x_i}$ where x_i is a particular training instance input to the unit, $\hat{f}(x)$ is the sigmoid unit output, and x^j denotes the j th input to the sigmoid unit. You may wish to use the notation x_i^j to refer to the j th component of x_i . Hint: The derivation is similar to the derivation of the BACKPROPAGATION training rule.
- 12.6.** Consider again the search trace of FOCL shown in Figure 12.8. Suppose that the hypothesis selected at the first level in the search is changed to

$$Cup \leftarrow \neg HasHandle$$

Describe the second-level candidate hypotheses that will be generated by FOCL as successors to this hypothesis. You need only include those hypotheses generated by FOCL's second search operator, which uses its domain theory. Don't forget to post-prune the sufficient conditions. Use the training data from Table 12.3.

- 12.7.** This chapter discussed three approaches to using prior knowledge to impact the search through the space of possible hypotheses. Discuss your ideas for how these three approaches could be integrated. Can you propose a specific algorithm that integrates at least two of these three for some specific hypothesis representation? What advantages and disadvantages would you anticipate from this integration?
- 12.8.** Consider again the question from Section 12.2.1, regarding what criterion to use for choosing among hypotheses when both data and prior knowledge are available. Give your own viewpoint on this issue.

REFERENCES

- Abu-Mostafa, Y. S. (1989). Learning from hints in neural networks. *Journal of Complexity*, 6(2), 192–198.
- Bergadano, F., & Giordana, A. (1990). Guiding induction with domain theories. In R. Michalski et al. (Eds.), *Machine learning: An artificial intelligence approach 3* (pp. 474–492). San Mateo, CA: Morgan Kaufmann.
- Bradshaw, G., Fozzard, R., & Cice, L. (1989). A connectionist expert system that really works. In *Advances in neural information processing*. San Mateo, CA: Morgan Kaufmann.
- Caruana, R. (1996). Algorithms and applications for multitask learning. *Proceedings of the 13th International Conference on Machine Learning*. San Francisco: Morgan Kaufmann.
- Cooper, G. C., & Herskovits, E. (1992). A Bayesian method for the induction of probabilistic networks from data. *Machine Learning*, 9, 309–347.
- Craven, M. W. (1996). *Extracting comprehensible models from trained neural networks* (PhD thesis) (UW Technical Report CS-TR-96-1326). Department of Computer Sciences, University of Wisconsin-Madison.

- Craven, M. W., & Shavlik, J. W. (1994). Using sampling and queries to extract rules from trained neural networks. *Proceedings of the 11th International Conference on Machine Learning* (pp. 37–45). San Mateo, CA: Morgan Kaufmann.
- Fu, L. M. (1989). Integration of neural heuristics into knowledge-based inference. *Connection Science*, 1(3), 325–339.
- Fu, L. M. (1993). Knowledge-based connectionism for revising domain theories. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(1), 173–182.
- Gallant, S. I. (1988). Connectionist expert systems. *CACM*, 31(2), 152–169.
- Koppel, M., Feldman, R., & Segre, A. (1994). Bias-driven revision of logical domain theories. *Journal of Artificial Intelligence*, 1, 159–208. <http://www.cs.washington.edu/research/jair/home.html>.
- Lacher, R., Hruska, S., & Kuncicky, D. (1991). *Backpropagation learning in expert networks* (Dept. of Computer Science Technical Report TR91-015). Florida State University, Tallahassee.
- Maclin, R., & Shavlik, J. (1993). Using knowledge-based neural networks to improve algorithms: Refining the Chou-Fasman algorithm for protein folding. *Machine Learning*, 11(3), 195–215.
- Mitchell, T. M., & Thrun, S. B. (1993a). Explanation-based neural network learning for robot control. In S. Hanson, J. Cowan, & C. Giles (Eds.), *Advances in neural information processing systems 5* (pp. 287–294). San Mateo, CA: Morgan-Kaufmann Press.
- Mitchell, T. M., & Thrun, S. B. (1993b). Explanation-based learning: A comparison of symbolic and neural network approaches. *Tenth International Conference on Machine Learning*, Amherst, MA.
- Mooney, R. (1993). Induction over the unexplained: Using overly-general domain theories to aid concept learning. *Machine Learning*, 10(1).
- O'Sullivan, J., Mitchell, T., & Thrun, S. (1997). Explanation-based learning for mobile robot perception. In K. Ikeuchi & M. Veloso (Eds.), *Symbolic Visual Learning* (pp. 295–324).
- Ourston, D., & Mooney, R. J. (1994). Theory refinement combining analytical and empirical methods. *Artificial Intelligence*, 66(2).
- Pazzani, M. J., & Brunk, C. (1993). Finding accurate frontiers: A knowledge-intensive approach to relational learning. *Proceedings of the 1993 National Conference on Artificial Intelligence* (pp. 328–334). AAAI Press.
- Pazzani, M. J., Brunk, C. A., & Silverstein, G. (1991). A knowledge-intensive approach to learning relational concepts. *Proceedings of the Eighth International Workshop on Machine Learning* (pp. 432–436). San Mateo, CA: Morgan Kaufmann.
- Pazzani, M. J., & Kibler, D. (1992). The utility of knowledge in inductive learning. *Machine Learning*, 9(1), 57–94.
- Pratt, L. Y. (1993a). *Transferring previously learned BACKPROPAGATION neural networks to new learning tasks* (Ph.D. thesis). Department of Computer Science, Rutgers University, New Jersey. (Also Rutgers Computer Science Technical Report ML-TR-37.)
- Pratt, L. Y. (1993b). Discriminability-based transfer among neural networks. In J. E. Moody et al. (Eds.), *Advances in Neural Information Processing Systems 5*. San Mateo, CA: Morgan Kaufmann.
- Rosenbloom, P. S., & Aasman, J. (1990). Knowledge level and inductive uses of chunking (ebl). *Proceedings of the Eighth National Conference on Artificial Intelligence* (pp. 821–827). AAAI Press.
- Russell, S., Binder, J., Koller, D., & Kanazawa, K. (1995). Local learning in probabilistic networks with hidden variables. *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, Montreal. Morgan Kaufmann.
- Shavlik, J., & Towell, G. (1989). An approach to combining explanation-based and neural learning algorithms. *Connection Science*, 1(3), 233–255.
- Simard, P. S., Victorri, B., LeCun, Y., & Denker, J. (1992). Tangent prop—A formalism for specifying selected invariances in an adaptive network. In J. Moody et al. (Eds.), *Advances in Neural Information Processing Systems 4*. San Mateo, CA: Morgan Kaufmann.
- Sudharth, S. C., & Holden, A. D. C. (1991). Symbolic-neural systems and the use of hints for developing complex systems. *International Journal of Man-Machine Studies*, 35(3), 291–311.

- Thrun, S. (1996). *Explanation based neural network learning: A lifelong learning approach*. Boston: Kluwer Academic Publishers.
- Thrun, S., & Mitchell, T. M. (1993). Integrating inductive neural network learning and explanation-based learning. *Proceedings of the 1993 International Joint Conference on Artificial Intelligence*.
- Thrun, S., & Mitchell, T. M. (1995). Learning one more thing. *Proceedings of the 1995 International Joint Conference on Artificial Intelligence*, Montreal.
- Towell, G., & Shavlik, J. (1989). An approach to combining explanation-based and neural learning algorithms. *Connection Science*, (1), 233–255.
- Towell, G., & Shavlik, J. (1994). Knowledge-based artificial neural networks. *Artificial Intelligence*, 70(1–2), 119–165.
- Towell, G., Shavlik, J., & Noordewier, M. (1990). Refinement of approximate domain theories by knowledge-based neural networks. *Proceedings of the Eighth National Conference on Artificial Intelligence* (pp. 861–866). Cambridge, MA: AAAI, MIT Press.
- Yang, Q., & Bhargava, V. (1990). Building expert systems by a modified perceptron network with rule-transfer algorithms (pp. 77–82). *International Joint Conference on Neural Networks*, IEEE.

CHAPTER 13

REINFORCEMENT LEARNING

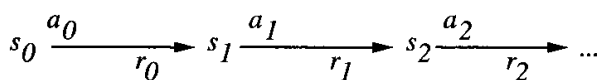
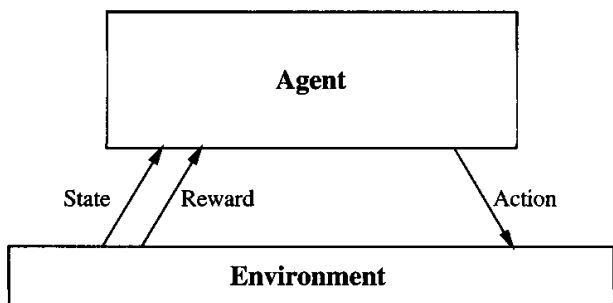
Reinforcement learning addresses the question of how an autonomous agent that senses and acts in its environment can learn to choose optimal actions to achieve its goals. This very generic problem covers tasks such as learning to control a mobile robot, learning to optimize operations in factories, and learning to play board games. Each time the agent performs an action in its environment, a trainer may provide a reward or penalty to indicate the desirability of the resulting state. For example, when training an agent to play a game the trainer might provide a positive reward when the game is won, negative reward when it is lost, and zero reward in all other states. The task of the agent is to learn from this indirect, delayed reward, to choose sequences of actions that produce the greatest cumulative reward. This chapter focuses on an algorithm called Q learning that can acquire optimal control strategies from delayed rewards, even when the agent has no prior knowledge of the effects of its actions on the environment. Reinforcement learning algorithms are related to dynamic programming algorithms frequently used to solve optimization problems.

13.1 INTRODUCTION

Consider building a learning robot. The robot, or *agent*, has a set of sensors to observe the *state* of its environment, and a set of *actions* it can perform to alter this state. For example, a mobile robot may have sensors such as a camera and sonars, and actions such as “move forward” and “turn.” Its task is to learn a control strategy, or *policy*, for choosing actions that achieve its goals. For example, the robot may have a goal of docking onto its battery charger whenever its battery level is low.

This chapter is concerned with how such agents can learn successful control policies by experimenting in their environment. We assume that the goals of the agent can be defined by a *reward* function that assigns a numerical value—an immediate payoff—to each distinct action the agent may take from each distinct state. For example, the goal of docking to the battery charger can be captured by assigning a positive reward (e.g., +100) to state-action transitions that immediately result in a connection to the charger and a reward of zero to every other state-action transition. This reward function may be built into the robot, or known only to an external teacher who provides the reward value for each action performed by the robot. The task of the robot is to perform sequences of actions, observe their consequences, and learn a control policy. The control policy we desire is one that, from any initial state, chooses actions that maximize the reward accumulated over time by the agent. This general setting for robot learning is summarized in Figure 13.1.

As is apparent from Figure 13.1, the problem of learning a control policy to maximize cumulative reward is very general and covers many problems beyond robot learning tasks. In general the problem is one of learning to control sequential processes. This includes, for example, manufacturing optimization problems in which a sequence of manufacturing actions must be chosen, and the reward to be maximized is the value of the goods produced minus the costs involved. It includes sequential scheduling problems such as choosing which taxis to send for passengers in a large city, where the reward to be maximized is a function of the wait time of the passengers and the total fuel costs of the taxi fleet. In general, we are interested in any type of agent that must learn to choose actions that alter the state of its environment and where a cumulative reward function is used to define the quality of any given action sequence. Within this class of problems we will consider specific settings, including settings in which the actions have deterministic or nondeterministic outcomes, and settings in which the agent



Goal: Learn to choose actions that maximize

$$r_0 + \gamma r_1 + \gamma^2 r_2 + \dots, \text{ where } 0 \leq \gamma < 1$$

FIGURE 13.1

An agent interacting with its environment. The agent exists in an environment described by some set of possible states S . It can perform any of a set of possible actions A . Each time it performs an action a_t in some state s_t the agent receives a real-valued reward r_t that indicates the immediate value of this state-action transition. This produces a sequence of states s_i , actions a_i , and immediate rewards r_i as shown in the figure. The agent's task is to learn a control policy, $\pi : S \rightarrow A$, that maximizes the expected sum of these rewards, with future rewards discounted exponentially by their delay.

has or does not have prior knowledge about the effects of its actions on the environment.

Note we have touched on the problem of learning to control sequential processes earlier in this book. In Section 11.4 we discussed explanation-based learning of rules to control search during problem solving. There the problem is for the agent to choose among alternative actions at each step in its search for some goal state. The techniques discussed here differ from those of Section 11.4, in that here we consider problems where the actions may have nondeterministic outcomes and where the learner lacks a domain theory that describes the outcomes of its actions. In Chapter 1 we discussed the problem of learning to choose actions while playing the game of checkers. There we sketched the design of a learning method very similar to those discussed in this chapter. In fact, one highly successful application of the reinforcement learning algorithms of this chapter is to a similar game-playing problem. Tesauro (1995) describes the TD-GAMMON program, which has used reinforcement learning to become a world-class backgammon player. This program, after training on 1.5 million self-generated games, is now considered nearly equal to the best human players in the world and has played competitively against top-ranked players in international backgammon tournaments.

The problem of learning a control policy to choose actions is similar in some respects to the function approximation problems discussed in other chapters. The target function to be learned in this case is a control policy, $\pi : S \rightarrow A$, that outputs an appropriate action a from the set A , given the current state s from the set S . However, this reinforcement learning problem differs from other function approximation tasks in several important respects.

- *Delayed reward.* The task of the agent is to learn a target function π that maps from the current state s to the optimal action $a = \pi(s)$. In earlier chapters we have always assumed that when learning some target function such as π , each training example would be a pair of the form $\langle s, \pi(s) \rangle$. In reinforcement learning, however, training information is not available in this form. Instead, the trainer provides only a sequence of immediate reward values as the agent executes its sequence of actions. The agent, therefore, faces the problem of *temporal credit assignment*: determining which of the actions in its sequence are to be credited with producing the eventual rewards.
- *Exploration.* In reinforcement learning, the agent influences the distribution of training examples by the action sequence it chooses. This raises the question of which experimentation strategy produces most effective learning. The learner faces a tradeoff in choosing whether to favor *exploration* of unknown states and actions (to gather new information), or *exploitation* of states and actions that it has already learned will yield high reward (to maximize its cumulative reward).
- *Partially observable states.* Although it is convenient to assume that the agent's sensors can perceive the entire state of the environment at each time step, in many practical situations sensors provide only partial information. For example, a robot with a forward-pointing camera cannot see what is

behind it. In such cases, it may be necessary for the agent to consider its previous observations together with its current sensor data when choosing actions, and the best policy may be one that chooses actions specifically to improve the observability of the environment.

- *Life-long learning.* Unlike isolated function approximation tasks, robot learning often requires that the robot learn several related tasks within the same environment, using the same sensors. For example, a mobile robot may need to learn how to dock on its battery charger, how to navigate through narrow corridors, and how to pick up output from laser printers. This setting raises the possibility of using previously obtained experience or knowledge to reduce sample complexity when learning new tasks.

13.2 THE LEARNING TASK

In this section we formulate the problem of learning sequential control strategies more precisely. Note there are many ways to do so. For example, we might assume the agent's actions are deterministic or that they are nondeterministic. We might assume that the agent can predict the next state that will result from each action, or that it cannot. We might assume that the agent is trained by an expert who shows it examples of optimal action sequences, or that it must train itself by performing actions of its own choice. Here we define one quite general formulation of the problem, based on Markov decision processes. This formulation of the problem follows the problem illustrated in Figure 13.1.

In a Markov decision process (MDP) the agent can perceive a set S of distinct states of its environment and has a set A of actions that it can perform. At each discrete time step t , the agent senses the current state s_t , chooses a current action a_t , and performs it. The environment responds by giving the agent a reward $r_t = r(s_t, a_t)$ and by producing the succeeding state $s_{t+1} = \delta(s_t, a_t)$. Here the functions δ and r are part of the environment and are not necessarily known to the agent. In an MDP, the functions $\delta(s_t, a_t)$ and $r(s_t, a_t)$ depend only on the current state and action, and not on earlier states or actions. In this chapter we consider only the case in which S and A are finite. In general, δ and r may be nondeterministic functions, but we begin by considering only the deterministic case.

The task of the agent is to learn a *policy*, $\pi : S \rightarrow A$, for selecting its next action a_t based on the current observed state s_t ; that is, $\pi(s_t) = a_t$. How shall we specify precisely which policy π we would like the agent to learn? One obvious approach is to require the policy that produces the greatest possible cumulative reward for the robot over time. To state this requirement more precisely, we define the cumulative value $V^\pi(s_t)$ achieved by following an arbitrary policy π from an arbitrary initial state s_t as follows:

$$\begin{aligned} V^\pi(s_t) &\equiv r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\ &\equiv \sum_{i=0}^{\infty} \gamma^i r_{t+i} \end{aligned} \tag{13.1}$$

where the sequence of rewards r_{t+i} is generated by beginning at state s_t and by repeatedly using the policy π to select actions as described above (i.e., $a_t = \pi(s_t)$, $a_{t+1} = \pi(s_{t+1})$, etc.). Here $0 \leq \gamma < 1$ is a constant that determines the relative value of delayed versus immediate rewards. In particular, rewards received i time steps into the future are discounted exponentially by a factor of γ^i . Note if we set $\gamma = 0$, only the immediate reward is considered. As we set γ closer to 1, future rewards are given greater emphasis relative to the immediate reward.

The quantity $V^\pi(s)$ defined by Equation (13.1) is often called the *discounted cumulative reward* achieved by policy π from initial state s . It is reasonable to discount future rewards relative to immediate rewards because, in many cases, we prefer to obtain the reward sooner rather than later. However, other definitions of total reward have also been explored. For example, *finite horizon* reward, $\sum_{i=0}^h r_{t+i}$, considers the undiscounted sum of rewards over a finite number h of steps. Another possibility is *average reward*, $\lim_{h \rightarrow \infty} \frac{1}{h} \sum_{i=0}^h r_{t+i}$, which considers the average reward per time step over the entire lifetime of the agent. In this chapter we restrict ourselves to considering discounted reward as defined by Equation (13.1). Mahadevan (1996) provides a discussion of reinforcement learning when the criterion to be optimized is average reward.

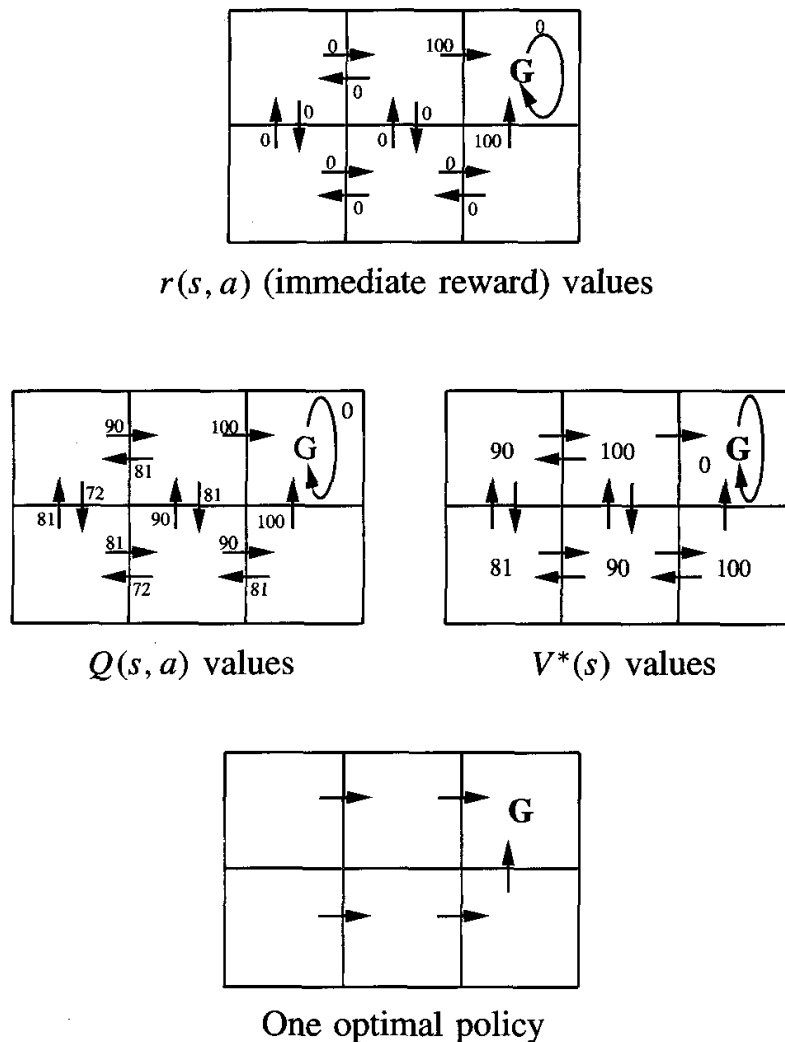
We are now in a position to state precisely the agent's learning task. We require that the agent learn a policy π that maximizes $V^\pi(s)$ for all states s . We will call such a policy an *optimal policy* and denote it by π^* .

$$\pi^* \equiv \underset{\pi}{\operatorname{argmax}} V^\pi(s), (\forall s) \quad (13.2)$$

To simplify notation, we will refer to the value function $V^{\pi^*}(s)$ of such an optimal policy as $V^*(s)$. $V^*(s)$ gives the maximum discounted cumulative reward that the agent can obtain starting from state s ; that is, the discounted cumulative reward obtained by following the optimal policy beginning at state s .

To illustrate these concepts, a simple grid-world environment is depicted in the topmost diagram of Figure 13.2. The six grid squares in this diagram represent six possible states, or locations, for the agent. Each arrow in the diagram represents a possible action the agent can take to move from one state to another. The number associated with each arrow represents the immediate reward $r(s, a)$ the agent receives if it executes the corresponding state-action transition. Note the immediate reward in this particular environment is defined to be zero for all state-action transitions except for those leading into the state labeled **G**. It is convenient to think of the state **G** as the goal state, because the only way the agent can receive reward, in this case, is by entering this state. Note in this particular environment, the only action available to the agent once it enters the state **G** is to remain in this state. For this reason, we call **G** an *absorbing* state.

Once the states, actions, and immediate rewards are defined, and once we choose a value for the discount factor γ , we can determine the optimal policy π^* and its value function $V^*(s)$. In this case, let us choose $\gamma = 0.9$. The diagram at the bottom of the figure shows one optimal policy for this setting (there are others as well). Like any policy, this policy specifies exactly one action that the

**FIGURE 13.2**

A simple deterministic world to illustrate the basic concepts of Q -learning. Each grid square represents a distinct state, each arrow a distinct action. The immediate reward function, $r(s, a)$ gives reward 100 for actions entering the goal state G , and zero otherwise. Values of $V^*(s)$ and $Q(s, a)$ follow from $r(s, a)$, and the discount factor $\gamma = 0.9$. An optimal policy, corresponding to actions with maximal Q values, is also shown.

agent will select in any given state. Not surprisingly, the optimal policy directs the agent along the shortest path toward the state G .

The diagram at the right of Figure 13.2 shows the values of V^* for each state. For example, consider the bottom right state in this diagram. The value of V^* for this state is 100 because the optimal policy in this state selects the “move up” action that receives immediate reward 100. Thereafter, the agent will remain in the absorbing state and receive no further rewards. Similarly, the value of V^* for the bottom center state is 90. This is because the optimal policy will move the agent from this state to the right (generating an immediate reward of zero), then upward (generating an immediate reward of 100). Thus, the discounted future reward from the bottom center state is

$$0 + \gamma 100 + \gamma^2 0 + \gamma^3 0 + \dots = 90$$

Recall that V^* is defined to be the sum of discounted future rewards over the infinite future. In this particular environment, once the agent reaches the absorbing state $\mathbf{G}$ its infinite future will consist of remaining in this state and receiving rewards of zero.

13.3 Q LEARNING

How can an agent learn an optimal policy π^* for an arbitrary environment? It is difficult to learn the function $\pi^* : S \rightarrow A$ directly, because the available training data does not provide training examples of the form $\langle s, a \rangle$. Instead, the only training information available to the learner is the sequence of immediate rewards $r(s_i, a_i)$ for $i = 0, 1, 2, \dots$. As we shall see, given this kind of training information it is easier to learn a numerical evaluation function defined over states and actions, then implement the optimal policy in terms of this evaluation function.

What evaluation function should the agent attempt to learn? One obvious choice is V^* . The agent should prefer state s_1 over state s_2 whenever $V^*(s_1) > V^*(s_2)$, because the cumulative future reward will be greater from s_1 . Of course the agent's policy must choose among actions, not among states. However, it can use V^* in certain settings to choose among actions as well. The optimal action in state s is the action a that maximizes the sum of the immediate reward $r(s, a)$ plus the value V^* of the immediate successor state, discounted by γ .

$$\pi^*(s) = \underset{a}{\operatorname{argmax}} [r(s, a) + \gamma V^*(\delta(s, a))] \quad (13.3)$$

(recall that $\delta(s, a)$ denotes the state resulting from applying action a to state s .) Thus, the agent can acquire the optimal policy by learning V^* , *provided it has perfect knowledge of the immediate reward function r and the state transition function δ* . When the agent knows the functions r and δ used by the environment to respond to its actions, it can then use Equation (13.3) to calculate the optimal action for any state s .

Unfortunately, learning V^* is a useful way to learn the optimal policy *only* when the agent has perfect knowledge of δ and r . This requires that it be able to perfectly predict the immediate result (i.e., the immediate reward and immediate successor) for every possible state-action transition. This assumption is comparable to the assumption of a perfect domain theory in explanation-based learning, discussed in Chapter 11. In many practical problems, such as robot control, it is impossible for the agent or its human programmer to predict in advance the exact outcome of applying an arbitrary action to an arbitrary state. Imagine, for example, the difficulty in describing δ for a robot arm shoveling dirt when the resulting state includes the positions of the dirt particles. In cases where either δ or r is unknown, learning V^* is unfortunately of no use for selecting optimal actions because the agent cannot evaluate Equation (13.3). What evaluation function should the agent use in this more general setting? The evaluation function Q , defined in the following section, provides one answer.

13.3.1 The Q Function

Let us define the evaluation function $Q(s, a)$ so that its value is the maximum discounted cumulative reward that can be achieved starting from state s and applying action a as the first action. In other words, the value of Q is the reward received immediately upon executing action a from state s , plus the value (discounted by γ) of following the optimal policy thereafter.

$$Q(s, a) \equiv r(s, a) + \gamma V^*(\delta(s, a)) \quad (13.4)$$

Note that $Q(s, a)$ is exactly the quantity that is maximized in Equation (13.3) in order to choose the optimal action a in state s . Therefore, we can rewrite Equation (13.3) in terms of $Q(s, a)$ as

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a) \quad (13.5)$$

Why is this rewrite important? Because it shows that if the agent learns the Q function instead of the V^* function, it will be able to select optimal actions *even when it has no knowledge of the functions r and δ* . As Equation (13.5) makes clear, it need only consider each available action a in its current state s and choose the action that maximizes $Q(s, a)$.

It may at first seem surprising that one can choose globally optimal action sequences by reacting repeatedly to the local values of Q for the current state. This means the agent can choose the optimal action without ever conducting a lookahead search to explicitly consider what state results from the action. Part of the beauty of Q learning is that the evaluation function is defined to have precisely this property—the value of Q for the current state and action summarizes in a single number all the information needed to determine the discounted cumulative reward that will be gained in the future if action a is selected in state s .

To illustrate, Figure 13.2 shows the Q values for every state and action in the simple grid world. Notice that the Q value for each state-action transition equals the r value for this transition plus the V^* value for the resulting state discounted by γ . Note also that the optimal policy shown in the figure corresponds to selecting actions with maximal Q values.

13.3.2 An Algorithm for Learning Q

Learning the Q function corresponds to learning the optimal policy. How can Q be learned?

The key problem is finding a reliable way to estimate training values for Q , given only a sequence of immediate rewards r spread out over time. This can be accomplished through iterative approximation. To see how, notice the close relationship between Q and V^* ,

$$V^*(s) = \max_{a'} Q(s, a')$$

which allows rewriting Equation (13.4) as

$$Q(s, a) = r(s, a) + \gamma \max_{a'} Q(\delta(s, a), a') \quad (13.6)$$

This recursive definition of Q provides the basis for algorithms that iteratively approximate Q (Watkins 1989). To describe the algorithm, we will use the symbol $\hat{Q}$ to refer to the learner's estimate, or hypothesis, of the actual Q function. In this algorithm the learner represents its hypothesis $\hat{Q}$ by a large table with a separate entry for each state-action pair. The table entry for the pair $\langle s, a \rangle$ stores the value for $\hat{Q}(s, a)$ —the learner's current hypothesis about the actual but unknown value $Q(s, a)$. The table can be initially filled with random values (though it is easier to understand the algorithm if one assumes initial values of zero). The agent repeatedly observes its current state s , chooses some action a , executes this action, then observes the resulting reward $r = r(s, a)$ and the new state $s' = \delta(s, a)$. It then updates the table entry for $\hat{Q}(s, a)$ following each such transition, according to the rule:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a') \quad (13.7)$$

Note this training rule uses the agent's current $\hat{Q}$ values for the new state s' to refine its estimate of $\hat{Q}(s, a)$ for the previous state s . This training rule is motivated by Equation (13.6), although the training rule concerns the agent's approximation $\hat{Q}$, whereas Equation (13.6) applies to the actual Q function. Note although Equation (13.6) describes Q in terms of the functions $\delta(s, a)$ and $r(s, a)$, the agent does not need to know these general functions to apply the training rule of Equation (13.7). Instead it executes the action in its environment and then observes the resulting new state s' and reward r . Thus, it can be viewed as sampling these functions at the current values of s and a .

The above Q learning algorithm for deterministic Markov decision processes is described more precisely in Table 13.1. Using this algorithm the agent's estimate $\hat{Q}$ converges in the limit to the actual Q function, provided the system can be modeled as a deterministic Markov decision process, the reward function r is

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.

Observe the current state s

Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$
-

TABLE 13.1

Q learning algorithm, assuming deterministic rewards and actions. The discount factor γ may be any constant such that $0 \leq \gamma < 1$.

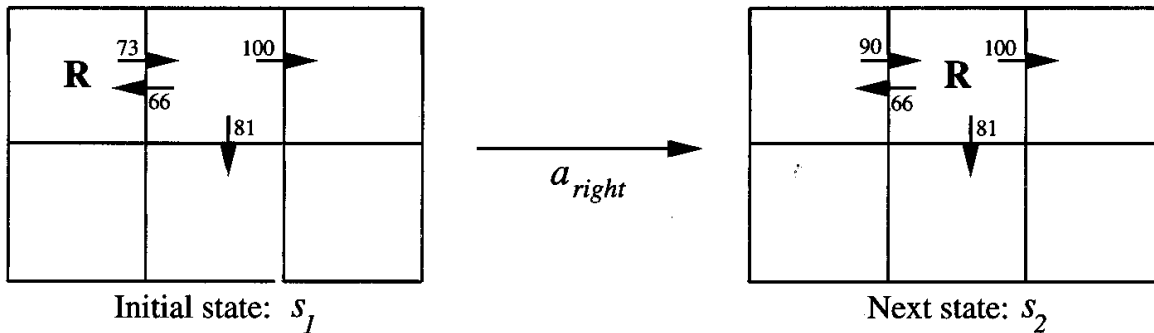
bounded, and actions are chosen so that every state-action pair is visited infinitely often.

13.3.3 An Illustrative Example

To illustrate the operation of the Q learning algorithm, consider a single action taken by an agent, and the corresponding refinement to $\hat{Q}$ shown in Figure 13.3. In this example, the agent moves one cell to the right in its grid world and receives an immediate reward of zero for this transition. It then applies the training rule of Equation (13.7) to refine its estimate $\hat{Q}$ for the state-action transition it just executed. According to the training rule, the new $\hat{Q}$ estimate for this transition is the sum of the received reward (zero) and the highest $\hat{Q}$ value associated with the resulting state (100), discounted by γ (.9).

Each time the agent moves forward from an old state to a new one, Q learning propagates $\hat{Q}$ estimates *backward* from the new state to the old. At the same time, the immediate reward received by the agent for the transition is used to augment these propagated values of $\hat{Q}$.

Consider applying this algorithm to the grid world and reward function shown in Figure 13.2, for which the reward is zero everywhere, except when entering the goal state. Since this world contains an absorbing goal state, we will assume that training consists of a series of *episodes*. During each episode, the agent begins at some randomly chosen state and is allowed to execute actions until it reaches the absorbing goal state. When it does, the episode ends and



$$\begin{aligned}
 \hat{Q}(s_1, a_{right}) &\leftarrow r + \gamma \max_{a'} \hat{Q}(s_2, a') \\
 &\leftarrow 0 + 0.9 \max\{66, 81, 100\} \\
 &\leftarrow 90
 \end{aligned}$$

FIGURE 13.3

The update to $\hat{Q}$ after executing a single action. The diagram on the left shows the initial state s_1 of the robot (R) and several relevant $\hat{Q}$ values in its initial hypothesis. For example, the value $\hat{Q}(s_1, a_{right}) = 72.9$, where a_{right} refers to the action that moves R to its right. When the robot executes the action a_{right} , it receives immediate reward $r = 0$ and transitions to state s_2 . It then updates its estimate $\hat{Q}(s_1, a_{right})$ based on its $\hat{Q}$ estimates for the new state s_2 . Here $\gamma = 0.9$.

the agent is transported to a new, randomly chosen, initial state for the next episode.

How will the values of $\hat{Q}$ evolve as the Q learning algorithm is applied in this case? With all the $\hat{Q}$ values initialized to zero, the agent will make no changes to any $\hat{Q}$ table entry until it happens to reach the goal state and receive a nonzero reward. This will result in refining the $\hat{Q}$ value for the single transition leading into the goal state. On the next episode, if the agent passes through this state adjacent to the goal state, its nonzero $\hat{Q}$ value will allow refining the value for some transition two steps from the goal, and so on. Given a sufficient number of training episodes, the information will propagate from the transitions with nonzero reward back through the entire state-action space available to the agent, resulting eventually in a $\hat{Q}$ table containing the Q values shown in Figure 13.2.

In the next section we prove that under certain assumptions the Q learning algorithm of Table 13.1 will converge to the correct Q function. First consider two general properties of this Q learning algorithm that hold for any deterministic MDP in which the rewards are non-negative, assuming we initialize all $\hat{Q}$ values to zero. The first property is that under these conditions the $\hat{Q}$ values never decrease during training. More formally, let $\hat{Q}_n(s, a)$ denote the learned $\hat{Q}(s, a)$ value after the n th iteration of the training procedure (i.e., after the n th state-action transition taken by the agent). Then

$$(\forall s, a, n) \quad \hat{Q}_{n+1}(s, a) \geq \hat{Q}_n(s, a)$$

A second general property that holds under these same conditions is that throughout the training process every $\hat{Q}$ value will remain in the interval between zero and its true Q value.

$$(\forall s, a, n) \quad 0 \leq \hat{Q}_n(s, a) \leq Q(s, a)$$

13.3.4 Convergence

Will the algorithm of Table 13.1 converge toward a $\hat{Q}$ equal to the true Q function? The answer is yes, under certain conditions. First, we must assume the system is a deterministic MDP. Second, we must assume the immediate reward values are bounded; that is, there exists some positive constant c such that for all states s and actions a , $|r(s, a)| < c$. Third, we assume the agent selects actions in such a fashion that it visits every possible state-action pair infinitely often. By this third condition we mean that if action a is a legal action from state s , then over time the agent must execute action a from state s repeatedly and with nonzero frequency as the length of its action sequence approaches infinity. Note these conditions are in some ways quite general and in others fairly restrictive. They describe a more general setting than illustrated by the example in the previous section, because they allow for environments with arbitrary positive or negative rewards, and for environments where any number of state-action transitions may produce nonzero rewards. The conditions are also restrictive in that they require the agent to visit every distinct state-action transition infinitely often. This is a very strong assumption in large (or continuous!) domains. We will discuss stronger

convergence results later. However, the result described in this section provides the basic intuition for understanding why Q learning works.

The key idea underlying the proof of convergence is that the table entry $\hat{Q}(s, a)$ with the largest error must have its error reduced by a factor of γ whenever it is updated. The reason is that its new value depends only in part on error-prone $\hat{Q}$ estimates, with the remainder depending on the error-free observed immediate reward r .

Theorem 13.1. Convergence of Q learning for deterministic Markov decision processes. Consider a Q learning agent in a deterministic MDP with bounded rewards $(\forall s, a)|r(s, a)| \leq c$. The Q learning agent uses the training rule of Equation (13.7), initializes its table $\hat{Q}(s, a)$ to arbitrary finite values, and uses a discount factor γ such that $0 \leq \gamma < 1$. Let $\hat{Q}_n(s, a)$ denote the agent's hypothesis $\hat{Q}(s, a)$ following the n th update. If each state-action pair is visited infinitely often, then $\hat{Q}_n(s, a)$ converges to $Q(s, a)$ as $n \rightarrow \infty$, for all s, a .

Proof. Since each state-action transition occurs infinitely often, consider consecutive intervals during which each state-action transition occurs at least once. The proof consists of showing that the maximum error over all entries in the $\hat{Q}$ table is reduced by at least a factor of γ during each such interval. $\hat{Q}_n$ is the agent's table of estimated Q values after n updates. Let Δ_n be the maximum error in $\hat{Q}_n$; that is

$$\Delta_n \equiv \max_{s,a} |\hat{Q}_n(s, a) - Q(s, a)|$$

Below we use s' to denote $\delta(s, a)$. Now for any table entry $\hat{Q}_n(s, a)$ that is updated on iteration $n + 1$, the magnitude of the error in the revised estimate $\hat{Q}_{n+1}(s, a)$ is

$$\begin{aligned} |\hat{Q}_{n+1}(s, a) - Q(s, a)| &= |(r + \gamma \max_{a'} \hat{Q}_n(s', a')) - (r + \gamma \max_{a'} Q(s', a'))| \\ &= \gamma |\max_{a'} \hat{Q}_n(s', a') - \max_{a'} Q(s', a')| \\ &\leq \gamma \max_{a'} |\hat{Q}_n(s', a') - Q(s', a')| \\ &\leq \gamma \max_{s'', a'} |\hat{Q}_n(s'', a') - Q(s'', a')| \end{aligned}$$

$$|\hat{Q}_{n+1}(s, a) - Q(s, a)| \leq \gamma \Delta_n$$

The third line above follows from the second line because for any two functions f_1 and f_2 the following inequality holds

$$|\max_a f_1(a) - \max_a f_2(a)| \leq \max_a |f_1(a) - f_2(a)|$$

In going from the third line to the fourth line above, note we introduce a new variable s'' over which the maximization is performed. This is legitimate because the maximum value will be at least as great when we allow this additional variable to vary. Note that by introducing this variable we obtain an expression that matches the definition of Δ_n .

Thus, the updated $\hat{Q}_{n+1}(s, a)$ for any s, a is at most γ times the maximum error in the $\hat{Q}_n$ table, Δ_n . The largest error in the initial table, Δ_0 , is bounded because values of $\hat{Q}_0(s, a)$ and $Q(s, a)$ are bounded for all s, a . Now after the first interval

during which each s, a is visited, the largest error in the table will be at most $\gamma \Delta_0$. After k such intervals, the error will be at most $\gamma^k \Delta_0$. Since each state is visited infinitely often, the number of such intervals is infinite, and $\Delta_n \rightarrow 0$ as $n \rightarrow \infty$. This proves the theorem. $\square$

13.3.5 Experimentation Strategies

Notice the algorithm of Table 13.1 does not specify how actions are chosen by the agent. One obvious strategy would be for the agent in state s to select the action a that maximizes $\hat{Q}(s, a)$, thereby exploiting its current approximation $\hat{Q}$. However, with this strategy the agent runs the risk that it will overcommit to actions that are found during early training to have high $\hat{Q}$ values, while failing to explore other actions that have even higher values. In fact, the convergence theorem above requires that each state-action transition occur infinitely often. This will clearly not occur if the agent always selects actions that maximize its current $\hat{Q}(s, a)$. For this reason, it is common in Q learning to use a probabilistic approach to selecting actions. Actions with higher $\hat{Q}$ values are assigned higher probabilities, but every action is assigned a nonzero probability. One way to assign such probabilities is

$$P(a_i|s) = \frac{k^{\hat{Q}(s, a_i)}}{\sum_j k^{\hat{Q}(s, a_j)}}$$

where $P(a_i|s)$ is the probability of selecting action a_i , given that the agent is in state s , and where $k > 0$ is a constant that determines how strongly the selection favors actions with high $\hat{Q}$ values. Larger values of k will assign higher probabilities to actions with above average $\hat{Q}$, causing the agent to *exploit* what it has learned and seek actions it believes will maximize its reward. In contrast, small values of k will allow higher probabilities for other actions, leading the agent to *explore* actions that do not currently have high $\hat{Q}$ values. In some cases, k is varied with the number of iterations so that the agent favors exploration during early stages of learning, then gradually shifts toward a strategy of exploitation.

13.3.6 Updating Sequence

One important implication of the above convergence theorem is that Q learning need not train on optimal action sequences in order to converge to the optimal policy. In fact, it can learn the Q function (and hence the optimal policy) while training from actions chosen completely at random at each step, as long as the resulting training sequence visits every state-action transition infinitely often. This fact suggests changing the sequence of training example transitions in order to improve training efficiency without endangering final convergence. To illustrate, consider again learning in an MDP with a single absorbing goal state, such as the one in Figure 13.1. Assume as before that we train the agent with a sequence of episodes. For each episode, the agent is placed in a random initial state and is allowed to perform actions and to update its $\hat{Q}$ table until it reaches the absorbing goal state. A new training episode is then begun by removing the agent from the

goal state and placing it at a new random initial state. As noted earlier, if we begin with all $\hat{Q}$ values initialized to zero, then after the first full episode only one entry in the agent's $\hat{Q}$ table will have been changed: the entry corresponding to the final transition into the goal state. Note that if the agent happens to follow the same sequence of actions from the same random initial state in its second full episode, then a second table entry would be made nonzero, and so on. If we run repeated identical episodes in this fashion, the frontier of nonzero $\hat{Q}$ values will creep backward from the goal state at the rate of one new state-action transition per episode. Now consider training on these same state-action transitions, but in reverse chronological order for each episode. That is, we apply the same update rule from Equation (13.7) for each transition considered, but perform these updates in reverse order. In this case, after the first full episode the agent will have updated its $\hat{Q}$ estimate for every transition along the path it took to the goal. This training process will clearly converge in fewer iterations, although it requires that the agent use more memory to store the entire episode before beginning the training for that episode.

A second strategy for improving the rate of convergence is to store past state-action transitions, along with the immediate reward that was received, and retrain on them periodically. Although at first it might seem a waste of effort to retrain on the same transition, recall that the updated $\hat{Q}(s, a)$ value is determined by the values $\hat{Q}(s', a)$ of the successor state $s' = \delta(s, a)$. Therefore, if subsequent training changes one of the $\hat{Q}(s', a)$ values, then retraining on the transition (s, a) may result in an altered value for $\hat{Q}(s, a)$. In general, the degree to which we wish to replay old transitions versus obtain new ones from the environment depends on the relative costs of these two operations in the specific problem domain. For example, in a robot domain with navigation actions that might take several seconds to perform, the delay in collecting a new state-action transition from the external world might be several orders of magnitude more costly than internally replaying a previously observed transition. This difference can be very significant given that Q learning can often require thousands of training iterations to converge.

Note throughout the above discussion we have kept our assumption that the agent does not know the state-transition function $\delta(s, a)$ used by the environment to create the successor state $s' = \delta(s, a)$, or the function $r(s, a)$ used to generate rewards. If it does know these two functions, then many more efficient methods are possible. For example, if performing external actions is expensive the agent may simply ignore the environment and instead simulate it internally, efficiently generating simulated actions and assigning the appropriate simulated rewards. Sutton (1991) describes the DYNA architecture that performs a number of simulated actions after each step executed in the external world. Moore and Atkeson (1993) describe an approach called *prioritized sweeping* that selects promising states to update next, focusing on predecessor states when the current state is found to have a large update. Peng and Williams (1994) describe a similar approach. A large number of efficient algorithms from the field of dynamic programming can be applied when the functions δ and r are known. Kaelbling et al. (1996) survey a number of these.

13.4 NONDETERMINISTIC REWARDS AND ACTIONS

Above we considered Q learning in deterministic environments. Here we consider the nondeterministic case, in which the reward function $r(s, a)$ and action transition function $\delta(s, a)$ may have probabilistic outcomes. For example, in Tesauro's (1995) backgammon playing program, action outcomes are inherently probabilistic because each move involves a roll of the dice. Similarly, in robot problems with noisy sensors and effectors it is often appropriate to model actions and rewards as nondeterministic. In such cases, the functions $\delta(s, a)$ and $r(s, a)$ can be viewed as first producing a probability distribution over outcomes based on s and a , and then drawing an outcome at random according to this distribution. When these probability distributions depend solely on s and a (e.g., they do not depend on previous states or actions), then we call the system a nondeterministic Markov decision process.

In this section we extend the Q learning algorithm for the deterministic case to handle nondeterministic MDPs. To accomplish this, we retrace the line of argument that led to the algorithm for the deterministic case, revising it where needed.

In the nondeterministic case we must first restate the objective of the learner to take into account the fact that outcomes of actions are no longer deterministic. The obvious generalization is to redefine the value V^π of a policy π to be the *expected value* (over these nondeterministic outcomes) of the discounted cumulative reward received by applying this policy

$$V^\pi(s_t) \equiv E \left[\sum_{i=0}^{\infty} \gamma^i r_{t+i} \right]$$

where, as before, the sequence of rewards r_{t+i} is generated by following policy π beginning at state s . Note this is a generalization of Equation (13.1), which covered the deterministic case.

As before, we define the optimal policy π^* to be the policy π that maximizes $V^\pi(s)$ for all states s . Next we generalize our earlier definition of Q from Equation (13.4), again by taking its expected value.

$$\begin{aligned} Q(s, a) &\equiv E[r(s, a) + \gamma V^*(\delta(s, a))] \\ &= E[r(s, a)] + \gamma E[V^*(\delta(s, a))] \\ &= E[r(s, a)] + \gamma \sum_{s'} P(s'|s, a) V^*(s') \end{aligned} \tag{13.8}$$

where $P(s'|s, a)$ is the probability that taking action a in state s will produce the next state s' . Note we have used $P(s'|s, a)$ here to rewrite the expected value of $V^*(\delta(s, a))$ in terms of the probabilities associated with the possible outcomes of the probabilistic δ .

As before we can re-express Q recursively

$$Q(s, a) = E[r(s, a)] + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q(s', a') \tag{13.9}$$

which is the generalization of the earlier Equation (13.6). To summarize, we have simply redefined $Q(s, a)$ in the nondeterministic case to be the expected value of its previously defined quantity for the deterministic case.

Now that we have generalized the definition of Q to accommodate the nondeterministic environment functions r and δ , a new training rule is needed. Our earlier training rule derived for the deterministic case (Equation 13.7) fails to converge in this nondeterministic setting. Consider, for example, a nondeterministic reward function $r(s, a)$ that produces different rewards each time the transition $\langle s, a \rangle$ is repeated. In this case, the training rule will repeatedly alter the values of $\hat{Q}(s, a)$, even if we initialize the $\hat{Q}$ table values to the correct Q function. In brief, this training rule does not converge. This difficulty can be overcome by modifying the training rule so that it takes a decaying weighted average of the current $\hat{Q}$ value and the revised estimate. Writing $\hat{Q}_n$ to denote the agent's estimate on the n th iteration of the algorithm, the following revised training rule is sufficient to assure convergence of $\hat{Q}$ to Q :

$$\hat{Q}_n(s, a) \leftarrow (1 - \alpha_n) \hat{Q}_{n-1}(s, a) + \alpha_n [r + \gamma \max_{a'} \hat{Q}_{n-1}(s', a')] \quad (13.10)$$

where

$$\alpha_n = \frac{1}{1 + \text{visits}_n(s, a)} \quad (13.11)$$

where s and a here are the state and action updated during the n th iteration, and where $\text{visits}_n(s, a)$ is the total number of times this state-action pair has been visited up to and including the n th iteration.

The key idea in this revised rule is that revisions to $\hat{Q}$ are made more gradually than in the deterministic case. Notice if we were to set α_n to 1 in Equation (13.10) we would have exactly the training rule for the deterministic case. With smaller values of α , this term is now averaged in with the current $\hat{Q}(s, a)$ to produce the new updated value. Notice that the value of α_n in Equation (13.11) decreases as n increases, so that updates become smaller as training progresses. By reducing α at an appropriate rate during training, we can achieve convergence to the correct Q function. The choice of α_n given above is one of many that satisfy the conditions for convergence, according to the following theorem due to Watkins and Dayan (1992).

Theorem 13.2. Convergence of Q learning for nondeterministic Markov decision processes. Consider a Q learning agent in a nondeterministic MDP with bounded rewards ($\forall s, a |r(s, a)| \leq c$). The Q learning agent uses the training rule of Equation (13.10), initializes its table $\hat{Q}(s, a)$ to arbitrary finite values, and uses a discount factor γ such that $0 \leq \gamma < 1$. Let $n(i, s, a)$ be the iteration corresponding to the i th time that action a is applied to state s . If each state-action pair is visited infinitely often, $0 \leq \alpha_n < 1$, and

$$\sum_{i=1}^{\infty} \alpha_{n(i, s, a)} = \infty, \quad \sum_{i=1}^{\infty} [\alpha_{n(i, s, a)}]^2 < \infty$$

then for all s and a , $\hat{Q}_n(s, a) \rightarrow Q(s, a)$ as $n \rightarrow \infty$, with probability 1.

While Q learning and related reinforcement learning algorithms can be proven to converge under certain conditions, in practice systems that use Q learning often require many thousands of training iterations to converge. For example, Tesauro's TD-GAMMON discussed earlier trained for 1.5 million backgammon games, each of which contained tens of state-action transitions.

13.5 TEMPORAL DIFFERENCE LEARNING

The Q learning algorithm learns by iteratively reducing the discrepancy between Q value estimates for adjacent states. In this sense, Q learning is a special case of a general class of *temporal difference* algorithms that learn by reducing discrepancies between estimates made by the agent at different times. Whereas the training rule of Equation (13.10) reduces the difference between the estimated $\hat{Q}$ values of a state and its immediate successor, we could just as well design an algorithm that reduces discrepancies between this state and more distant descendants or ancestors.

To explore this issue further, recall that our Q learning training rule calculates a training value for $\hat{Q}(s_t, a_t)$ in terms of the values for $\hat{Q}(s_{t+1}, a_{t+1})$ where s_{t+1} is the result of applying action a_t to the state s_t . Let $Q^{(1)}(s_t, a_t)$ denote the training value calculated by this one-step lookahead

$$Q^{(1)}(s_t, a_t) \equiv r_t + \gamma \max_a \hat{Q}(s_{t+1}, a)$$

One alternative way to compute a training value for $Q(s_t, a_t)$ is to base it on the observed rewards for two steps

$$Q^{(2)}(s_t, a_t) \equiv r_t + \gamma r_{t+1} + \gamma^2 \max_a \hat{Q}(s_{t+2}, a)$$

or, in general, for n steps

$$Q^{(n)}(s_t, a_t) \equiv r_t + \gamma r_{t+1} + \cdots + \gamma^{(n-1)} r_{t+n-1} + \gamma^n \max_a \hat{Q}(s_{t+n}, a)$$

Sutton (1988) introduces a general method for blending these alternative training estimates, called $TD(\lambda)$. The idea is to use a constant $0 \leq \lambda \leq 1$ to combine the estimates obtained from various lookahead distances in the following fashion

$$Q^\lambda(s_t, a_t) \equiv (1 - \lambda) \left[Q^{(1)}(s_t, a_t) + \lambda Q^{(2)}(s_t, a_t) + \lambda^2 Q^{(3)}(s_t, a_t) + \cdots \right]$$

An equivalent recursive definition for Q^λ is

$$Q^\lambda(s_t, a_t) = r_t + \gamma \left[(1 - \lambda) \max_a \hat{Q}(s_t, a_t) + \lambda Q^\lambda(s_{t+1}, a_{t+1}) \right]$$

Note if we choose $\lambda = 0$ we have our original training estimate $Q^{(1)}$, which considers only one-step discrepancies in the $\hat{Q}$ estimates. As λ is increased, the algorithm places increasing emphasis on discrepancies based on more distant lookaheads. At the extreme value $\lambda = 1$, only the observed r_{t+i} values are considered,

with no contribution from the current $\hat{Q}$ estimate. Note when $\hat{Q} = Q$, the training values given by Q^λ will be identical for all values of λ such that $0 \leq \lambda \leq 1$.

The motivation for the TD(λ) method is that in some settings training will be more efficient if more distant lookaheads are considered. For example, when the agent follows an optimal policy for choosing actions, then Q^λ with $\lambda = 1$ will provide a perfect estimate for the true Q value, regardless of any inaccuracies in $\hat{Q}$. On the other hand, if action sequences are chosen suboptimally, then the r_{t+i} observed far into the future can be misleading.

Peng and Williams (1994) provide a further discussion and experimental results showing the superior performance of Q^λ in one problem domain. Dayan (1992) shows that under certain assumptions a similar TD(λ) approach applied to learning the V^* function converges correctly for any λ such that $0 \leq \lambda \leq 1$. Tesauro (1995) uses a TD(λ) approach in his TD-GAMMON program for playing backgammon.

13.6 GENERALIZING FROM EXAMPLES

Perhaps the most constraining assumption in our treatment of Q learning up to this point is that the target function is represented as an explicit lookup table, with a distinct table entry for every distinct input value (i.e., state-action pair). Thus, the algorithms we discussed perform a kind of rote learning and make no attempt to estimate the Q value for unseen state-action pairs by generalizing from those that have been seen. This rote learning assumption is reflected in the convergence proof, which proves convergence only if every possible state-action pair is visited (infinitely often!). This is clearly an unrealistic assumption in large or infinite spaces, or when the cost of executing actions is high. As a result, more practical systems often combine function approximation methods discussed in other chapters with the Q learning training rules described here.

It is easy to incorporate function approximation algorithms such as BACKPROPAGATION into the Q learning algorithm, by substituting a neural network for the lookup table and using each $\hat{Q}(s, a)$ update as a training example. For example, we could encode the state s and action a as network inputs and train the network to output the target values of $\hat{Q}$ given by the training rules of Equations (13.7) and (13.10). An alternative that has sometimes been found to be more successful in practice is to train a separate network for each action, using the state as input and $\hat{Q}$ as output. Another common alternative is to train one network with the state as input, but with one $\hat{Q}$ output for each action. Recall that in Chapter 1, we discussed approximating an evaluation function over checkerboard states using a linear function and the LMS algorithm.

In practice, a number of successful reinforcement learning systems have been developed by incorporating such function approximation algorithms in place of the lookup table. Tesauro's successful TD-GAMMON program for playing backgammon used a neural network and the BACKPROPAGATION algorithm together with a TD(λ) training rule. Zhang and Dietterich (1996) use a similar combination of BACKPROPAGATION and TD(λ) for job-shop scheduling tasks. Crites and Barto (1996) describe

a neural network reinforcement learning approach for an elevator scheduling task. Thrun (1996) reports a neural network based approach to Q learning to learn basic control procedures for a mobile robot with sonar and camera sensors. Mahadevan and Connell (1991) describe a Q learning approach based on clustering states, applied to a simple mobile robot control problem.

Despite the success of these systems, for other tasks reinforcement learning fails to converge once a generalizing function approximator is introduced. Examples of such problematic tasks are given by Boyan and Moore (1995), Baird (1995), and Gordon (1995). Note the convergence theorems discussed earlier in this chapter apply only when $\hat{Q}$ is represented by an explicit table. To see the difficulty, consider using a neural network rather than an explicit table to represent $\hat{Q}$. Note if the learner updates the network to better fit the training Q value for a particular transition $\langle s_i, a_i \rangle$, the altered network weights may also change the $\hat{Q}$ estimates for arbitrary other transitions. Because these weight changes may increase the error in $\hat{Q}$ estimates for these other transitions, the argument proving the original theorem no longer holds. Theoretical analyses of reinforcement learning with generalizing function approximators are given by Gordon (1995) and Tsitsiklis (1994). Baird (1995) proposes gradient-based methods that circumvent this difficulty by directly minimizing the sum of squared discrepancies in estimates between adjacent states (also called Bellman residual errors).

13.7 RELATIONSHIP TO DYNAMIC PROGRAMMING

Reinforcement learning methods such as Q learning are closely related to a long line of research on dynamic programming approaches to solving Markov decision processes. This earlier work has typically assumed that the agent possesses perfect knowledge of the functions $\delta(s, a)$ and $r(s, a)$ that define the agent's environment. Therefore, it has primarily addressed the question of how to compute the optimal policy using the least computational effort, assuming the environment could be perfectly simulated and no direct interaction was required. The novel aspect of Q learning is that it assumes the agent does *not* have knowledge of $\delta(s, a)$ and $r(s, a)$, and that instead of moving about in an internal mental model of the state space, it must move about the real world and observe the consequences. In this latter case our primary concern is usually the number of real-world actions that the agent must perform to converge to an acceptable policy, rather than the number of computational cycles it must expend. The reason is that in many practical domains such as manufacturing problems, the costs in time and in dollars of performing actions in the external world dominate the computational costs. Systems that learn by moving about the real environment and observing the results are typically called *online* systems, whereas those that learn solely by simulating actions within an internal model are called *offline* systems.

The close correspondence between these earlier approaches and the reinforcement learning problems discussed here is apparent by considering Bellman's equation, which forms the foundation for many dynamic programming approaches

to solving MDPs. Bellman's equation is

$$(\forall s \in S) V^*(s) = E[r(s, \pi(s)) + \gamma V^*(\delta(s, \pi(s)))]$$

Note the very close relationship between Bellman's equation and our earlier definition of an optimal policy in Equation (13.2). Bellman (1957) showed that the optimal policy π^* satisfies the above equation and that any policy π satisfying this equation is an optimal policy. Early work on dynamic programming includes the Bellman-Ford shortest path algorithm (Bellman 1958; Ford and Fulkerson 1962), which learns paths through a graph by repeatedly updating the estimated distance to the goal for each graph node, based on the distances for its neighbors. In this algorithm the assumption that graph edges and the goal node are known is equivalent to our assumption that $\delta(s, a)$ and $r(s, a)$ are known. Barto et al. (1995) discuss the close relationship between reinforcement learning and dynamic programming.

13.8 SUMMARY AND FURTHER READING

The key points discussed in this chapter include:

- Reinforcement learning addresses the problem of learning control strategies for autonomous agents. It assumes that training information is available in the form of a real-valued reward signal given for each state-action transition. The goal of the agent is to learn an action policy that maximizes the total reward it will receive from any starting state.
- The reinforcement learning algorithms addressed in this chapter fit a problem setting known as a Markov decision process. In Markov decision processes, the outcome of applying any action to any state depends only on this action and state (and not on preceding actions or states). Markov decision processes cover a wide range of problems including many robot control, factory automation, and scheduling problems.
- Q learning is one form of reinforcement learning in which the agent learns an evaluation function over states and actions. In particular, the evaluation function $Q(s, a)$ is defined as the maximum expected, discounted, cumulative reward the agent can achieve by applying action a to state s . The Q learning algorithm has the advantage that it can be employed even when the learner has no prior knowledge of how its actions affect its environment.
- Q learning can be proven to converge to the correct Q function under certain assumptions, when the learner's hypothesis $\hat{Q}(s, a)$ is represented by a lookup table with a distinct entry for each $\langle s, a \rangle$ pair. It can be shown to converge in both deterministic and nondeterministic MDPs. In practice, Q learning can require many thousands of training iterations to converge in even modest-sized problems.
- Q learning is a member of a more general class of algorithms, called temporal difference algorithms. In general, temporal difference algorithms learn

by iteratively reducing the discrepancies between the estimates produced by the agent at different times.

- Reinforcement learning is closely related to dynamic programming approaches to Markov decision processes. The key difference is that historically these dynamic programming approaches have assumed that the agent possesses knowledge of the state transition function $\delta(s, a)$ and reward function $r(s, a)$. In contrast, reinforcement learning algorithms such as Q learning typically assume the learner lacks such knowledge.

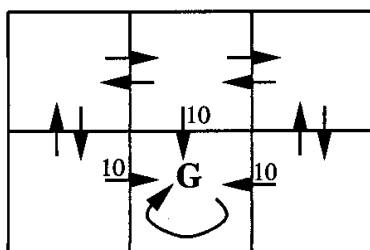
The common theme that underlies much of the work on reinforcement learning is to iteratively reduce the discrepancy between evaluations of successive states. Some of the earliest work on such methods is due to Samuel (1959). His checkers learning program attempted to learn an evaluation function for checkers by using evaluations of later states to generate training values for earlier states. Around the same time, the Bellman-Ford, single-destination, shortest-path algorithm was developed (Bellman 1958; Ford and Fulkerson 1962), which propagated distance-to-goal values from nodes to their neighbors. Research on optimal control led to the solution of Markov decision processes using similar methods (Bellman 1961; Blackwell 1965). Holland's (1986) bucket brigade method for learning classifier systems used a similar method for propagating credit in the face of delayed rewards. Barto et al. (1983) discussed an approach to temporal credit assignment that led to Sutton's paper (1988) defining the $TD(\lambda)$ method and proving its convergence for $\lambda = 0$. Dayan (1992) extended this result to arbitrary values of λ . Watkins (1989) introduced Q learning to acquire optimal policies when the reward and action transition functions are unknown. Convergence proofs are known for several variations on these methods. In addition to the convergence proofs presented in this chapter see, for example, (Baird 1995; Bertsekas 1987; Tsitsiklis 1994, Singh and Sutton 1996).

Reinforcement learning remains an active research area. McCallum (1995) and Littman (1996), for example, discuss the extension of reinforcement learning to settings with hidden state variables that violate the Markov assumption. Much current research seeks to scale up these methods to larger, more practical problems. For example, Maclin and Shavlik (1996) describe an approach in which a reinforcement learning agent can accept imperfect advice from a trainer, based on an extension to the KBANN algorithm (Chapter 12). Lin (1992) examines the role of teaching by providing suggested action sequences. Methods for scaling up by employing a hierarchy of actions are suggested by Singh (1993) and Lin (1993). Dietterich and Flann (1995) explore the integration of explanation-based methods with reinforcement learning, and Mitchell and Thrun (1993) describe the application of the EBNN algorithm (Chapter 12) to Q learning. Ring (1994) explores continual learning by the agent over multiple tasks.

Recent surveys of reinforcement learning are given by Kaelbling et al. (1996); Barto (1992); Barto et al. (1995); Dean et al. (1993).

EXERCISES

- 13.1. Give a second optimal policy for the problem illustrated in Figure 13.2.
- 13.2. Consider the deterministic grid world shown below with the absorbing goal-state **G**. Here the immediate rewards are 10 for the labeled transitions and 0 for all unlabeled transitions.
- Give the V^* value for every state in this grid world. Give the $Q(s, a)$ value for every transition. Finally, show an optimal policy. Use $\gamma = 0.8$.
 - Suggest a change to the reward function $r(s, a)$ that alters the $Q(s, a)$ values, but does not alter the optimal policy. Suggest a change to $r(s, a)$ that alters $Q(s, a)$ but does not alter $V^*(s, a)$.
 - Now consider applying the Q learning algorithm to this grid world, assuming the table of $\hat{Q}$ values is initialized to zero. Assume the agent begins in the bottom left grid square and then travels clockwise around the perimeter of the grid until it reaches the absorbing goal state, completing the first training episode. Describe which $\hat{Q}$ values are modified as a result of this episode, and give their revised values. Answer the question again assuming the agent now performs a second identical episode. Answer it again for a third episode.



- 13.3. Consider playing Tic-Tac-Toe against an opponent who plays randomly. In particular, assume the opponent chooses with uniform probability any open space, unless there is a forced move (in which case it makes the obvious correct move).
- Formulate the problem of learning an optimal Tic-Tac-Toe strategy in this case as a Q -learning task. What are the states, transitions, and rewards in this non-deterministic Markov decision process?
 - Will your program succeed if the opponent plays optimally rather than randomly?
- 13.4. Note in many MDPs it is possible to find two policies π_1 and π_2 such that π_1 outperforms π_2 if the agent begins in some state s_1 , but π_2 outperforms π_1 if it begins in some other state s_2 . Put another way, $V^{\pi_1}(s_1) > V^{\pi_2}(s_1)$, but $V^{\pi_2}(s_2) > V^{\pi_1}(s_2)$. Explain why there will always exist a single policy that maximizes $V^\pi(s)$ for every initial state s (i.e., an optimal policy π^*). In other words, explain why an MDP always allows a policy π^* such that $(\forall \pi, s) V^{\pi^*}(s) \geq V^\pi(s)$.

REFERENCES

- Baird, L. (1995). Residual algorithms: Reinforcement learning with function approximation. *Proceedings of the Twelfth International Conference on Machine Learning* (pp. 30–37). San Francisco: Morgan Kaufmann.

- Barto, A. (1992). Reinforcement learning and adaptive critic methods. In D. White & S. Sofge (Eds.), *Handbook of intelligent control: Neural, fuzzy, and adaptive approaches* (pp. 469–491). New York: Van Nostrand Reinhold.
- Barto, A., Bradtke, S., & Singh, S. (1995). Learning to act using real-time dynamic programming. *Artificial Intelligence*, Special volume: Computational research on interaction and agency, 72(1), 81–138.
- Barto, A., Sutton, R., & Anderson, C. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 13(5), 834–846.
- Bellman, R. E. (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.
- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87–90.
- Bellman, R. (1961). *Adaptive control processes*. Princeton, NJ: Princeton University Press.
- Berenji, R. (1992). Learning and tuning fuzzy controllers through reinforcements. *IEEE Transactions on Neural Networks*, 3(5), 724–740.
- Bertsekas, D. (1987). *Dynamic programming: Deterministic and stochastic models*. Englewood Cliffs, NJ: Prentice Hall.
- Blackwell, D. (1965). Discounted dynamic programming. *Annals of Mathematical Statistics*, 36, 226–235.
- Boyan, J., & Moore, A. (1995). Generalization in reinforcement learning: Safely approximating the value function. In G. Tesauro, D. Touretzky, & T. Leen (Eds.), *Advances in Neural Information Processing Systems 7*. Cambridge, MA: MIT Press.
- Crites, R., & Barto, A. (1996). Improving elevator performance using reinforcement learning. In D. S. Touretzky, M. C. Mozer, & M. C. Hasselmo (Eds.), *Advances in Neural Information Processing Systems*, 8.
- Dayan, P. (1992). The convergence of TD(λ) for general λ . *Machine Learning*, 8, 341–362.
- Dean, T., Basye, K., & Shewchuk, J. (1993). Reinforcement learning for planning and control. In S. Minton (Ed.), *Machine Learning Methods for Planning* (pp. 67–92). San Francisco: Morgan Kaufmann.
- Dietterich, T. G., & Flann, N. S. (1995). Explanation-based learning and reinforcement learning: A unified view. *Proceedings of the 12th International Conference on Machine Learning* (pp. 176–184). San Francisco: Morgan Kaufmann.
- Ford, L., & Fulkerson, D. (1962). *Flows in networks*. Princeton, NJ: Princeton University Press.
- Gordon, G. (1995). Stable function approximation in dynamic programming. *Proceedings of the Twelfth International Conference on Machine Learning* (pp. 261–268). San Francisco: Morgan Kaufmann.
- Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of AI Research*, 4, 237–285. Online journal at <http://www.cs.washington.edu/research/jair/home.html>.
- Holland, J. H. (1986). Escaping brittleness: The possibilities of general-purpose learning algorithms applied to parallel rule-based systems. In Michalski, Carbonell, & Mitchell (Eds.), *Machine learning: An artificial intelligence approach* (Vol. 2, pp. 593–623). San Francisco: Morgan Kaufmann.
- Laird, J. E., & Rosenbloom, P. S. (1990). Integrating execution, planning, and learning in SOAR for external environments. *Proceedings of the Eighth National Conference on Artificial Intelligence* (pp. 1022–1029). Menlo Park, CA: AAAI Press.
- Lin, L. J. (1992). Self-improving reactive agents based on reinforcement learning, planning, and teaching. *Machine Learning*, 8, 293–321.
- Lin, L. J. (1993). Hierarchical learning of robot skills by reinforcement. *Proceedings of the International Conference on Neural Networks*.
- Littman, M. (1996). *Algorithms for sequential decision making* (Ph.D. dissertation and Technical Report CS-96-09). Brown University, Department of Computer Science, Providence, RI.
- Maclin, R., & Shavlik, J. W. (1996). Creating advice-taking reinforcement learners. *Machine Learning*, 22, 251–281.

- Mahadevan, S. (1996). Average reward reinforcement learning: Foundations, algorithms, and empirical results. *Machine Learning*, 22(1), 159–195.
- Mahadevan, S., & Connell, J. (1991). Automatic programming of behavior-based robots using reinforcement learning. In *Proceedings of the Ninth National Conference on Artificial Intelligence*. San Francisco: Morgan Kaufmann.
- McCallum, A. (1995). *Reinforcement learning with selective perception and hidden state* (Ph.D. dissertation). Department of Computer Science, University of Rochester, Rochester, NY.
- Mitchell, T. M., & Thrun, S. B. (1993). Explanation-based neural network learning for robot control. In C. Giles, S. Hanson, & J. Cowan (Eds.), *Advances in Neural Information Processing Systems* 5 (pp. 287–294). San Francisco: Morgan-Kaufmann.
- Moore, A., & Atkeson C. (1993). Prioritized sweeping: Reinforcement learning with less data and less real time. *Machine Learning*, 13, 103.
- Peng, J., & Williams, R. (1994). Incremental multi-step *Q*-learning. *Proceedings of the Eleventh International Conference on Machine Learning* (pp. 226–232). San Francisco: Morgan Kaufmann.
- Ring, M. (1994). *Continual learning in reinforcement environments* (Ph.D. dissertation). Computer Science Department, University of Texas at Austin, Austin, TX.
- Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3, 211–229.
- Singh, S. (1992). Reinforcement learning with a hierarchy of abstract models. *Proceedings of the Tenth National Conference on Artificial Intelligence* (pp. 202–207). San Jose, CA: AAAI Press.
- Singh, S. (1993). *Learning to solve markovian decision processes* (Ph.D. dissertation). Also CMPSCI Technical Report 93-77, Department of Computer Science, University of Massachusetts at Amherst.
- Singh, S., & Sutton, R. (1996). Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22, 123.
- Sutton, R. (1988). Learning to predict by the methods of temporal differences. *Machine learning*, 3, 9–44.
- Sutton R. (1991). Planning by incremental dynamic programming. *Proceedings of the Eighth International Conference on Machine Learning* (pp. 353–357). San Francisco: Morgan Kaufmann.
- Tesauro, G. (1995). Temporal difference learning and TD-GAMMON. *Communications of the ACM*, 38(3), 58–68.
- Thrun, S. (1992). The role of exploration in learning control. In D. White & D. Sofge (Eds.), *Handbook of intelligent control: Neural, fuzzy, and adaptive approaches* (pp. 527–559). New York: Van Nostrand Reinhold.
- Thrun, S. (1996). *Explanation-based neural network learning: A lifelong learning approach*. Boston: Kluwer Academic Publishers.
- Tsitsiklis, J. (1994). Asynchronous stochastic approximation and *Q*-learning. *Machine Learning*, 16(3), 185–202.
- Watkins, C. (1989). *Learning from delayed rewards* (Ph.D. dissertation). King's College, Cambridge, England.
- Watkins, C., & Dayan, P. (1992). *Q*-learning. *Machine Learning*, 8, 279–292.
- Zhang, W., & Dietterich, T. G. (1996). High-performance job-shop scheduling with a time-delay TD(λ) network. In D. S. Touretzky, M. C. Mozer, & M. E. Hasselmo (Eds.), *Advances in neural information processing systems*, 8, 1024–1030.

APPENDIX

NOTATION

Below is a summary of notation used in this book.

(a, b]: Brackets of the form [,], (, and) are used to represent intervals, where square brackets represent intervals including the boundary and round parentheses represent intervals excluding the boundary. For example, (1, 3] represents the interval $1 < x \leq 3$.

$\sum_{i=1}^n x_i$: The sum $x_1 + x_2 + \cdots + x_n$.

$\prod_{i=1}^n x_i$: The product $x_1 \cdot x_2 \cdots x_n$.

$\vdash$: The symbol for logical entailment. For example, $A \vdash B$ denotes that B follows deductively from A .

$>_g$: The symbol for the *more general than* relation. For example, $h_i >_g h_j$ denotes that hypothesis h_i is more general than h_j .

$\operatorname{argmax}_{x \in X} f(x)$: The value of x that maximizes $f(x)$. For example,

$$\operatorname{argmax}_{x \in \{1, 2, -3\}} x^2 = -3$$

$\hat{f}(x)$: A function that approximates the function $f(x)$.

δ : In PAC-learning, a bound on the probability of failure. In artificial neural network learning, the error term associated with a single unit output.

- ϵ : A bound on the error of a hypothesis (in PAC-learning).
- η : The learning rate in neural network and related learning methods.
- μ : The mean of a probability distribution.
- σ : The standard deviation of a probability distribution.
- $\nabla E(\vec{w})$: The gradient of E with respect to the vector $\vec{w}$.
- C : Class of possible target functions.
- D : The training data.
- $\mathcal{D}$: A probability distribution over the instance space.
- $E[x]$: The expected value of x .
- $E(\vec{w})$: The sum of squared errors of an artificial neural network whose weights are given by the vector $\vec{w}$.
- Error*: The error in a discrete-valued hypothesis or prediction.
- H : Hypothesis space.
- $h(x)$: The prediction produced by hypothesis h for instance x .
- $P(x)$: The probability (mass) of x .
- $\text{Pr}(x)$: The probability (mass) of the event x .
- $p(x)$: The probability density of x .
- $Q(s, a)$: The Q function from reinforcement learning.
- $\Re$: The set of real numbers.
- $VC(H)$: The Vapnik-Chervonenkis dimension of the hypothesis space H .
- $VS_{H,D}$: The Version Space; that is, the set of hypotheses from H that are consistent with D .
- w_{ji} : In artificial neural networks, the weight from node i to node j .
- X : Instance space.

INDEXES

SUBJECT INDEX

Page numbers in *italics* refer to tables; numbers in **bold** to figures. An “*n*” following a page number refers to a footnote on that page.

A

Absorbing state, 371
 ABSTRIPS, 329
 Acyclic neural networks. *See* Multilayer feedforward networks
 Adaline rule. *See* Delta rule
 Additive Chernoff bounds, 210–211
 Adelines, 123
 Agents, in reinforcement learning, 368
 Agnostic learning, 210–211, 225
 ALVINN system, 82–83, **84**
 Analytical-inductive learning. *See* Inductive-analytical learning
 Analytical learning, 307–330
 inductive learning, comparison with, 310, 328–329, 334–336, 362
 ANN learning. *See* Neural network learning
 ANNs. *See* Neural networks, artificial
 Antecedents of Horn clause, 285
 AQ algorithm, 279–280
 AQ14 algorithm, comparison with GABIL, 256, 258
 Arbitrary functions, representation by feedforward networks, 105–106
 Artificial intelligence, influence on machine learning, 4
 Artificial neural networks. *See* Neural networks, artificial
 ASSISTANT, 77
 Astronomical structures, machine learning classification of, 3
 Attributes:
 choice of, in sequential vs. simultaneous covering algorithms, 280–281
 continuous-valued, 72–73
 cost-sensitive measures, 75–76
 discrete-valued, 72
 measures for selection of, 73–74, 77
 missing values, strategies for, 75

Autonomous vehicles, 3, 4, 82–83, **84**
 Average reward, 371

B

Backgammon learning program. *See* TD-GAMMON
 BACKPROPAGATION algorithm, 83, 97, 124
 applications of, 81, **84**, 85, 96, 113
 convergence and local minima, 104–105
 definition of, 98
 discovery of hidden layer representations, 106–109, 123
 feedforward networks as hypothesis space, 105–106
 gradient descent search, 89, 115–116, 123
 inductive bias of, 106
 KBANN algorithm:
 comparison with, 344–345
 use in, 339
 momentum, addition of, 100, 104
 overfitting in, 108, 110–111
 in *Q* learning, 384
 search of hypothesis space, 97, 106, 122–123
 in decision tree learning, comparison with, 106
 by genetic algorithms, comparison with, 259
 by KBANN and TANGENTPROP algorithms, comparison with, 350–351
 stochastic gradient descent version, 98–100, 104–105, 107–108
 TANGENTPROP algorithm, comparison with, 349
 weight update rule:
 alternative error functions, 117–118
 derivation of, 101–102
 for hidden unit weights, 103

- in KBANN algorithm, 343–344
 - optimization methods, 119
 - for output unit weights, 102–103, 171
- Backtracking, ID3 algorithm and, 62
- Backward chaining search for explanation generation, 314
- Baldwin effect, 250, 267
 - computational models for, 267–268
- Bayes classifier, naive. *See* Naive Bayes classifier
- Bayes optimal classifier, 174–176, 197, 222
 - learning Boolean concepts using version spaces, 176
- Bayes optimal learner. *See* Bayes optimal classifier
- Bayes rule. *See* Bayes theorem
- Bayes theorem, 4, 156–159
 - in BRUTE-FORCE MAP LEARNING algorithm, 160–162
 - concept learning and, 158–163
 - in inductive-analytical learning, 338
- Bayesian belief networks, 184–191
 - choice among alternative networks, 190
 - conditional independence in, 185
 - constraint-based approaches in, 191
 - gradient ascent search in, 188–190
 - inference methods, 187–188
 - joint probability distribution representation, 185–187
 - learning from training data, 188–191
 - naive Bayes classifier, comparison with, 186
 - representation of causal knowledge, 187
- Bayesian classifiers, 198. *See also* Bayes optimal classifier; Naive Bayes classifier
- Bayesian learning, 154–198
 - decision tree learning, comparison with, 198
- Bayesian methods, influence on machine learning, 4
- Beam search:
 - general-to-specific. *See* General-to-specific beam search
 - generate-and-test. *See* Generate-and-test beam search
- Bellman-Ford shortest path algorithm, 386, 387
- Bellman residual errors, 385
- Bellman's equation, 385–386
- BFS-ID3 algorithm, 63
- Binomial distribution, 133–137, 143, 151
- Biological evolution, 249, 250, 266–267
- Biological neural networks, comparison with artificial neural networks, 82
- Bit strings, 252–253, 258–259, 269
- Blocks, stacking of. *See* Stacking problems
- Body of Horn clause, 285
- Boolean conjunctions, PAC learning of, 211–212
- Boolean functions:
 - representation by feedforward networks, 105–106
 - representation by perceptrons, 87–88
- Boundary set representation for version spaces, 31–36
 - definition of, 31
- Bounds:
 - one-sided, 141, 144
 - two-sided, 141
- Brain, neural activity in, 82
- Breadth first search in ID3 algorithm, 63
- BRUTE-FORCE MAP LEARNING algorithm, 159–162
 - Bayes theorem in, 160–162
- C
- C4.5 algorithm, 55, 77
 - GABIL, comparison with, 256, 258
 - missing attribute values, method for handling, 75
 - rule post-pruning in, 71–72
- CADET system, 241–244
- CANDIDATE-ELIMINATION algorithm, 29–37, 45–47
 - applications of, 29, 302
 - Bayesian interpretation of, 163
 - computation of version spaces, 32–36
 - definition of, 33
 - ID3 algorithm, comparison with, 61–64
 - inductive bias of, 43–46, 63–64
 - limitations of, 29, 37, 41, 42, 46
 - search of hypothesis space, 64
- Candidate specializations:
 - generated by FOCL algorithm, 357–361
 - generated by FOIL algorithm, 287–288, 357–358

- CART system, 77
- CASCADE-CORRELATION algorithm, 121–123
- Case-based reasoning, 231, 240–244, 246, 247
 - advantages of, 243–244
 - applications of, 240
 - other instance-based learning methods, comparison with, 240
- Causal knowledge, representation by Bayesian belief networks, 187
- Central Limit Theorem, 133, 142–143, 167
- Checkers learning program, 2–3, 5–14, 387
 - algorithms for, 14
 - design, 13
 - as sequential control process, 369
- Chemical mass spectroscopy, CANDIDATE-ELIMINATION algorithm in, 29
- Chess learning program, 308–310
 - explanation-based learning in, 325
- Chunking, 327, 330
- CIGOL, 302
- Circuit design, genetic programming in, 265–266
- Circuit layout, genetic algorithms in, 256
- Classification problems, 54
- CLASSIFY_NAIVE_BAYES_TEXT, 182–183
- CLAUDIEN, 302
- Clauses, 284, 285
- CLS. *See* Concept Learning System
- Clustering, 191
- CN2 algorithm, 278, 301
 - choice of attribute-pairs in, 280–281
- Complexity, sample. *See* Sample complexity
- Computational complexity, 202
- Computational complexity theory, influence on machine learning, 4
- Computational learning theory, 201–227
- Concept learning, 20–47
 - algorithms for, 47
 - Bayes theorem and, 158–163
 - definition of, 21
 - genetic algorithms in, 256
 - ID3 algorithm specialized for, 56
 - notation for, 22–23
 - search of hypothesis space, 23–25, 46–47
 - task design in, 21–22
- Concept Learning System, 77
- Concepts, partially learned, 38–39
- Conditional independence, 185
 - in Bayesian belief networks, 186–187
- Confidence intervals, 133, 138–141, 150, 151
 - for discrete-valued hypotheses, 131–132, 140–141
 - derivation of, 142–143
 - one-sided, 144, 145
- Conjugate gradient method, 119
- Conjunction of boolean literals, PAC learning of, 211–212
- Consequent of Horn clause, 285
- Consistent learners, 162–163
 - bound on sample complexity, 207–210, 225
 - equation for, 209
- Constants, in logic, 284, 285
- Constraint-based approaches in Bayesian belief networks, 191
- Constructive induction, 292
- Continuous functions, representation by feedforward networks, 105–106
- Continuous-valued hypotheses, training error of, 89–90
- Continuous-valued target function, 197
 - maximum likelihood (ML) hypothesis for, 164–167
- Control theory, influence on machine learning, 4
- Convergence of Q learning algorithm:
 - in deterministic environments, 377–380, 386
 - in nondeterministic environments, 382–383, 386
- Credit assignment, 5
- Critic, 12, 13
- Cross entropy, 170
 - minimization of, 118
- Cross-validation, 111–112
 - for comparison of learning algorithms, 145–151
 - k -fold. *See* k -fold cross-validation
 - in k -NEAREST NEIGHBOR algorithm, 235

leave-one-out, 235
 in neural network learning, 111–112
 Crossover mask, 254
 Crossover operators, 252–254, 261,
 262
 single-point, 254, 261
 two-point, 254, 257–258
 uniform, 255
 Crowding, 259
 Cumulative reward, 371
 Curse of dimensionality, 235

D

Data mining, 17
 Decision tree learning, 52–77
 algorithms for, 55, 77. *See also* C4.5
 algorithm; ID3 algorithm
 applications of, 54
 Bayesian learning, comparison with, 198
 impact of pruning on accuracy, 128–129
 inductive bias in, 63–66
 k-NEAREST NEIGHBOR algorithm,
 comparison with, 235
 Minimum Description Length principle
 in, 173–174
 neural network learning, comparison
 with, 85
 overfitting in, 67–69, 76–77, 111
 post-pruning in, 68–69, 77
 reduced-error pruning in, 69–71
 rule post-pruning in, 71–72, 281
 search of hypothesis space, 60–62
 by BACKPROPAGATION algorithm,
 comparison with, 106
 Deductive learning, 321–322
 Degrees of freedom, 147
 Delayed learning methods, comparison
 with eager learning, 244–245
 Delayed reward, in reinforcement learning,
 369
 Delta rule, 11, 88–90, 94, 99, 123
 Demes, 268
 Determinations, 325
 Deterministic environments, *Q* learning
 algorithm for, 375
 Directed acyclic neural networks. *See*
 Multilayer feedforward networks
 Discounted cumulative reward, 371

Discrete-valued hypotheses:
 confidence intervals for, 131–132,
 140–141
 derivation of, 142–143
 training error of, 205
 Discrete-valued target functions,
 approximation by decision tree
 learning, 52
 Disjunctive sets of rules, learning by
 sequential covering algorithms,
 275–276
 Distance-weighted *k*-NEAREST NEIGHBOR
 algorithm, 233–234
 Domain-independent learning algorithms,
 336
 Domain theory, 310, 329. *See also*
 Imperfect domain theory; Perfect
 domain theory; Prior knowledge
 in analytical learning, 311–312
 as KBANN neural network, 342–343
 in PROLOG-EBG, 322
 weighting of components in EBNN,
 351–352
 DYNA, 380
 Dynamic programming:
 applications to reinforcement learning,
 380
 reinforcement learning and, 385–387

E

Eager learning methods, comparison with
 lazy learning, 244–245
 EBG algorithm, 313
 EBNN algorithm, 351–356, 362, 387
 other explanation-based learning
 methods, comparison with, 356
 prior knowledge and gradient descent in,
 339
 TANGENTPROP algorithm in, 353
 weighting of inductive-analytical
 components in, 355, 362
 EGGS algorithm, 313
 EM algorithm, 190–196, 197
 applications of, 191, 194
 derivation of algorithm for *k*-means,
 195–196
 search for maximum likelihood (ML)
 hypothesis, 194–195

- Entailment, 321*n*
 - relationship with θ -subsumption and more-general-than partial ordering, 299–300
 - Entropy, 55–57, 282
 - of optimal code, 172*n*
 - Environment, in reinforcement learning, 368
 - Equivalent sample size, 179–180
 - Error bars for discrete-valued hypotheses.
 - See* Confidence intervals, for discrete-valued hypotheses
 - Error of hypotheses:
 - sample. *See* Sample error
 - training. *See* Training error
 - true. *See* True error
 - Estimation bias, 133, 137–138, 151
 - Estimator, 133, 137–138, 143, 150–151
 - Evolution of populations:
 - argument for Occam's razor, 66
 - in genetic algorithms, 260–262
 - Evolutionary computation, 250, 262
 - applications of, 269
 - Example-driven search, comparison with generate-and-test beam search, 281
 - Expected value, 133, 136
 - Experiment generator, 12–13
 - Explanation-based learning, 312–330
 - applications of, 325–328
 - derivation of new features, 320–321
 - inductive bias in, 322–323
 - inductive learning and, 330
 - lazy methods in, 328
 - limitations of, 308, 329
 - prior knowledge in, 308–309
 - reinforcement learning and, 330
 - utility analysis in, 327–328
 - Explanations generated by backward chaining search, 314
 - Explicit prior knowledge, 329
 - Exploration in reinforcement learning, 369
- F**
- Face recognition, 17
 - BACKPROPAGATION algorithm in, 81, 112–117
 - Feedforward networks. *See* Multilayer feedforward networks
 - FIND-S algorithm, 26–28, 46
 - Bayesian interpretation of, 162–163
 - definition of, 26
 - inductive bias of, 45
 - limitations of, 28–29
 - mistake-bound learning in, 220–221
 - PAC learning of boolean conjunctions with, 212
 - search of hypothesis space, 27–28
 - Finite horizon reward, 371
 - First-order Horn clauses, 283–284, 318–319. *See also* First-order rules
 - in analytical learning, 311
 - in PROLOG-EBG, 313, 314
 - First-order logic, basic definitions, 285
 - First-order representations, applications of, 275
 - First-order resolution rule, 296–297
 - First-order rules, 274–275, 283, 301, 302.
 - See also* First-order Horn clauses
 - in FOIL algorithm, 285–291
 - propositional rules, comparison with, 283
 - Fitness function, 250–252, 255–256, 258
 - Fitness proportionate selection, 255
 - Fitness sharing, 259
 - FOCL algorithm, 302
 - extensions to FOIL, 357
 - search step alteration with prior knowledge, 339–340
 - FOIL algorithm, 286, 290–291, 302
 - extensions in FOCL, 357
 - information gain measure in, 289
 - LEARN-ONE-RULE and sequential covering algorithms, comparison with, 287
 - learning first-order rules in, 285–291
 - post-pruning in, 291
 - recursive rule learning in, 290
 - Function approximation, 8
 - Function approximation algorithms:
 - choice of, 9–11
 - as lookup table substitute, 384
 - Functions, in logic, 284, 285
- G**
- GABIL, 256–259, 269
 - C4.5 and AQ14 algorithms, comparison with, 256, 258

- extensions to, 258–259
 - ID5R algorithm, comparison with, 258
 - Gain ratio, 73–74
 - GAs. *See* Genetic algorithms
 - Gaussian distribution. *See* Normal distribution
 - Gaussian kernel function, 238–240
 - General-to-specific beam search, 277–279, 302
 - advantages of, 281
 - in CN2 algorithm, 278
 - in FOCL algorithm, 357–361
 - in FOIL algorithm, 287, 357–358
 - General-to-specific ordering of
 - hypotheses, 24–25, 45–46. *See also* More-general-than partial ordering
 - Generalization accuracy in neural networks, 110–111
 - Generalizer, 12, 13
 - Generate-and-test beam search, 250
 - example-driven search, comparison with, 281
 - inverse entailment operators, comparison with, 299
 - inverse resolution, comparison with, 298–299
 - Genetic algorithms, 249–270
 - advantages of, 250
 - applications of, 256, 269
 - fitness function in, 255–256
 - limitations of, 259
 - parallelization of, 268
 - representation of hypotheses, 252–253
 - search of hypothesis space, 259, 268–269
 - Genetic operators, 252–255, 257, 261–262
 - Genetic programming, 250, 262–266, 269
 - applications of, 265, 269
 - performance of, 266
 - representation in, 262–263
 - Gibbs algorithm, 176
 - Global method, 234
 - GOLEM, 281
 - GP. *See* Genetic programming
 - Gradient ascent search, 170–171
 - in Bayesian belief networks, 188–190
 - Gradient descent search, 89–91, 93, 97, 115–116, 123
 - in EBNN algorithm, 339
 - least-squared error hypothesis in, 167
 - limitations of, 92
 - weight update rule, 91–92, 237
 - stochastic approximation to, 92–94, 98–100, 104–105, 107–108
 - Gradient of error, 91
 - Greedy search:
 - in sequential covering algorithms, 276–278
 - in PROLOG-EBG, 323
 - GRENDEL program, 303
 - Ground literal, 285
- ## H
- HALVING algorithm, 223
 - mistake-bound learning in, 221–222
 - Handwriting recognition, 3–4
 - BACKPROPAGATION algorithm in, 81
 - TANGENTPROP algorithm in, 348–349
 - Head of Horn clause, 285
 - Hidden layer representations, discovery
 - by BACKPROPAGATION algorithm, 106–109, 123
 - Hidden units:
 - BACKPROPAGATION weight tuning rule for, 103
 - CASCADE-CORRELATION algorithm, addition by, 121–123
 - choice in radial basis function networks, 239–240
 - in face recognition task, 115–117
 - Hill-climbing search:
 - in FOIL algorithm, 286, 287
 - in genetic algorithms, 268
 - in ID3 algorithm, 60–61
 - Hoeffding bounds, 210–211
 - Horn clauses, 284, 285
 - Horn clauses, first-order. *See* First-order Horn clauses
 - Human learning:
 - explanations in, 309
 - prior knowledge in, 330
 - Hypotheses. *See also* Discrete-valued hypotheses; General-to-specific ordering of hypotheses; Hypothesis space
 - error differences between two, 143–144
 - estimation of accuracy, 129–130

Hypotheses, estimation of accuracy
 (*continued*)
 bias and variance in estimate, 129, 151, 152
 errors in, 129–131, 151
 evaluation of, 128–129
 justification of, in inductive vs. analytical learning, 334–336
 representations of, 23
 testing of, 144–145
 Hypothesis space, 14–15
 bias in, 40–42, 46, 129
 finite, sample complexity for, 207–214, 225
 infinite, sample complexity for, 214–220
 VC dimension of, 214–217
 Hypothesis space search
 by BACKPROPAGATION algorithm, 97, 106, 122–123
 comparison with decision tree learning, 106
 comparison with KBANN and TANGENTPROP algorithms, 350–351
 by CANDIDATE-ELIMINATION algorithm, 64
 in concept learning, 23–25, 46–47
 constraints on, 302–303
 by FIND-S algorithm, 27–28
 by FOIL algorithm, 286–287, 357–361
 by genetic algorithms, 250, 259
 by gradient descent, 90–91
 by ID3 algorithm, 60–62, 64, 76
 by KBANN algorithm, 346
 by learning algorithms, 24
 by LEARN-ONE-RULE, 277
 in machine learning, 14–15, 18
 use of prior knowledge, 339–340, 362

I

ID3 algorithm, 55–64, 77
 backtracking and, 62
 CANDIDATE-ELIMINATION algorithm, comparison with, 61–62
 choice of attributes in, 280–281
 choice of decision tree, 63
 cost-sensitive measures, 75–76
 extensions to, 77. *See also* C4.5 algorithm

inductive bias of, 63–64, 76
 LEARN-ONE-RULE, search comparison with, 277
 limitations of, 61–62
 overfitting in, 67–68
 search of hypothesis space, 60–62, 64, 76
 sequential covering algorithms, comparison with, 280–281
 specialized for concept learning, 56
 use of information gain in, 58–60
 ID5R algorithm, comparison with GABIL, 258
 ILP. *See* Inductive logic programming
 Image encoding in face recognition, 114
 Imperfect domain theory:
 in EBNN algorithm, 356
 in explanation-based learning, 330
 in FOCL algorithm, 360
 in KBANN algorithm, 344–345
 Incremental explanation methods, 328
 Incremental gradient descent. *See* Stochastic gradient descent
 INCREMENTAL VERSION SPACE MERGING algorithm, 47
 Inductive-analytical learning, 334–363
 advantages of, 362
 explanation-based learning and, 330
 learning problem, 337–338
 prior knowledge methods to alter search, 339–340, 362
 properties of ideal systems, 337
 weighting of components in EBNN algorithm, 351–352, 355
 weighting prior knowledge in, 338
 Inductive bias, 39–45, 137–138. *See also* Occam's razor; Preference bias; Restriction bias
 of BACKPROPAGATION algorithm, 106
 bias-free learning, 40–42
 of CANDIDATE-ELIMINATION algorithm, 43–46, 63–64
 in decision tree learning, 63–66
 definition of, 43
 in explanation-based learning, 322–323
 of FIND-S algorithm, 45
 of ID3 algorithm, 63–64, 76
 of inductive learning algorithms, 42–46
 of *k*-NEAREST NEIGHBOR algorithm, 234

of LMS algorithm, 64
 of ROTE-LEARNER algorithm, 44–45
 Inductive inference. *See* Inductive learning
 Inductive learning, 42, 307–308. *See also* Decision tree learning; Genetic algorithms; Inductive logic programming; Neural network learning
 analytical learning, comparison with, 310, 328–329, 334–336, 362
 inductive bias in, 42–46
 Inductive learning hypothesis, 23
 Inductive logic programming, 275, 291
 PROLOG-EBG, comparison with, 322
 Information gain, 73
 definition of, 57–58
 in FOIL algorithm, 289
 in ID3 algorithm, 55, 58–60
 Information theory:
 influence on machine learning, 4
 Minimum Description Length principle and, 172
 Initialize-the-hypothesis approach, 339–346
 Bayesian belief networks in, 346
 Instance-based learning, 230–247. *See also* Case-based reasoning; *k*-NEAREST NEIGHBOR algorithm; Locally weighted regression
 advantages, 245–246
 case-based reasoning, comparison with other methods, 240
 limitations of, 231
 Inverse entailment, 292, 302
 first-order, 297
 generate-and-test beam search, comparison with, 299
 in PROGOL, 300–302
 Inverse resolution, 294–296, 302
 first-order, 297–298
 generate-and-test beam search, comparison with, 298–299
 limitations of, 300
 Inverted deduction, 291–293

J

Jacobian, 354
 Job-shop scheduling, genetic algorithms in, 256

Joint probability distribution, in Bayesian belief networks, 185–187

K

k-fold cross-validation, 112, 147, 150
k-means problem, 191–193
 derivation of EM algorithm for, 195–196
k-NEAREST NEIGHBOR algorithm, 231–233, 246
 applications of, 234
 cross-validation in, 235
 decision tree and rule learning, comparison with, 235
 distance-weighted, 233–234
 inductive bias of, 234
 memory indexing in, 236
k-term CNF expressions, 213–214
k-term DNF expressions, 213–214
 K2 algorithm, 190–191
 KBANN algorithm, 340–347, 362, 387
 advantages of, 344
 BACKPROPAGATION algorithm, comparison with, 344–345
 BACKPROPAGATION weight update rule in, 343–344
 hypothesis space search by BACKPROPAGATION and TANGENTPROP, comparison with, 350–351
 limitations of, 345
 prior knowledge in, 339
kd-tree, 236
 Kernel function, 236, 238, 246
 Kernel function, Gaussian. *See* Gaussian kernel function
 Knowledge-Based Artificial Neural Network (KBANN) algorithm. *See* KBANN algorithm
 Knowledge compilation, 320
 Knowledge level learning, 323–325
 Knowledge reformulation, 320

L

Lamarckian evolution, 266
 Language bias. *See* Restriction bias
 Lazy explanation methods, 328
 Lazy learning methods, comparison with eager learning, 244–245
 LEARN_NAIVE_BAYES_TEXT, 182–183

- LEARN-ONE-RULE algorithm:
 FOIL algorithm, comparison with, 287
 ID3 algorithm, search comparison with, 277
 rule performance in, 282
 rule post-pruning in, 281
 variations of, 279–280, 286
- Learning:
 human. *See* Human learning
 machine. *See* Machine learning
- Learning algorithms
 consistent learners, 162–163
 design of, 9–11, 17
 domain-independent, 336
 error differences between two, 145–151
 search of hypothesis space, 24
- Learning problems, 2–5, 17
 computational theory of, 201–202
 in inductive-analytical learning, 337–338
- Learning rate, 88, 91
- Learning systems:
 design of, 5–14, 17
 program modules, 11–14
- Least mean squares algorithm. *See* LMS algorithm
- Least-squared error hypothesis:
 classifiers for, 198
 gradient descent in, 167
 maximum likelihood (ML) hypothesis and, 164–167
- Leave-one-out cross-validation, 235
- Legal case reasoning, case-based reasoning in, 240
- LEMMA-ENUMERATOR algorithm, 324
- Lifelong learning, 370
- Line search, 119
- Linear programming, as weight update algorithm, 95
- Linearly separable sets, 86, 89, 95
- LIST-THEN-ELIMINATE algorithm, 30
- Literal, 284, 285
- LMS algorithm, 11, 15
 inductive bias of, 64
- LMS weight update rule. *See* Delta rule
- Local method, 234
- Locally weighted regression, 231, 236–238, 246
 limitations of, 238
 weight update rules in, 237–238
- Logical constants, 284, 285
- Logical terms, 284, 285
- Logistic function, 96, 104
- Lookup table:
 function approximation algorithms as substitute, 384
 neural network as substitute, 384
- Lower bound on sample complexity, 217–218
- ## M
- m*-estimate of probability, 179–180, 198, 282
- Machine learning, 15. *See also* entries beginning with Learning
 applications, 3, 17
 definition of, 2
 influence of other disciplines on, 4, 17
 search of hypothesis space, 14–15, 18
- Manufacturing process control, 17
- MAP hypothesis. *See* Maximum a posteriori hypothesis
- MAP LEARNING algorithm, BRUTE-FORCE. *See* BRUTE-FORCE MAP LEARNING algorithm
- Markov decision processes (MDP), 370, 387
 applications of, 386
- MARKUS, 302
- MARVIN, 302
- Maximally general hypotheses,
 computation by CANDIDATE-ELIMINATION algorithm, 31, 46
- Maximally specific hypotheses:
 computation by CANDIDATE-ELIMINATION algorithm, 31, 46
 computation by FIND-S algorithm, 26–28, 62–63
- Maximum a posteriori (MAP) hypothesis, 157, 197. *See also* BRUTE-FORCE MAP LEARNING algorithm
 naive Bayes classifier and, 178
 output of consistent learners, 162–163
- Maximum likelihood (ML) hypothesis, 157
 EM algorithm search for, 194–195
 least-squared error hypothesis and, 164–167

prediction of probabilities with,
 167–170
 MDP. *See* Markov decision processes
 Mean error, 143
 Mean value, 133, 136
 Means-ends planner, 326
 Mechanical design, case-based reasoning
 in, 240–244
 Medical diagnosis:
 attribute selection measure, 76
 Bayes theorem in, 157–158
 META-DENDRAL, 302
 MFOIL, 302
 Minimum Description Length principle,
 66, 69, 171–173, 197, 198
 in decision tree learning, 173–174
 in inductive logic programming,
 292–293
 MIS, 302
 Mistake-bound learning, 202, 220, 226
 in CANDIDATE-ELIMINATION algorithm,
 221–222
 in FIND-S algorithm, 220–221
 in HALVING algorithm, 221–222
 in LIST-THEN-ELIMINATE algorithm,
 221–222
 in WEIGHTED-MAJORITY algorithm,
 224–225
 Mistake bounds, optimal. *See* Optimal
 mistake bounds
 ML hypothesis. *See* Maximum likelihood
 hypothesis
 Momentum, addition to BACKPROPAGATION
 algorithm, 100, 104
 More-general-than partial ordering, 24–28,
 46
 in CANDIDATE-ELIMINATION algorithm,
 29
 in FIND-S algorithm, 26–28
 θ -subsumption, entailment, and, 299–300
 in version spaces, 31
 Multilayer feedforward networks
 BACKPROPAGATION algorithm in, 95–101
 function representation in, 105–106, 115
 representation of decision surfaces, 96
 training of multiple networks, 105
 VC dimension of, 218–220
 Mutation operator, 252, 253, 255, 257, 262

N

Naive Bayes classifier, 154–155, 177–179,
 197
 Bayesian belief network, comparison
 with, 186
 maximum a posteriori (MAP) hypothesis
 and, 178
 use in text classification, 180–184
 Naive Bayes learner. *See* Naive Bayes
 classifier
 Negation-as-failure strategy, 279, 319,
 321*n*
 Negative literal, 284, 285
 Neural network learning, 81–124.
 See also BACKPROPAGATION
 algorithm; CASCADE-CORRELATION
 algorithm; EBNN algorithm;
 KBANN algorithm; TANGENTPROP
 algorithm
 applications of, 83, 85
 in face recognition, 113
 cross-validation in, 111–112
 decision tree learning, comparison with,
 85
 discovery of hidden layer representations
 in, 107
 overfitting in, 123
 in Q learning, 384
 representation in, 82–83, 84, 105–106
 Neural networks, artificial, 81–124.
 See also Multilayer feedforward
 networks; Radial basis function
 networks; Recurrent networks
 biological neural networks, comparison
 with, 82
 creation by KBANN algorithm, 342–343
 VC dimension of, 218–220
 Neural networks, biological, 82
 Neurobiology, influence on machine
 learning, 4, 82
 New features:
 derivation in BACKPROPAGATION
 algorithm, 106–109, 123
 derivation in explanation-based learning,
 320–321
 NEWSWEEDER system, 183–184
 Nondeterministic environments, Q learning
 in, 381–383

Normal distribution, 133, 139–140, 143,
151, 165
for noise, 167
in paired tests, 149

O

Occam's razor, 4, 65–66, 171
Offline learning systems, 385
One-sided bounds, 141, 144
Online learning systems, 385
Optimal brain damage approach, 122
Optimal code, 172
Optimal mistake bounds, 222–223
Optimal policy for selecting actions,
371–372
Optimization problems:
 explanation-based learning in, 325
 genetic algorithms in, 256, 269
 reinforcement learning in, 256
Output encoding in face recognition,
114–115
Output units, BACKPROPAGATION weight
 update rule for, 102–103
Overfitting, 123
 in BACKPROPAGATION algorithm, 108,
 110–111
 in decision tree learning, 66–69, 76–77,
 111
 definition of, 67
 Minimum Description Length principle
 and, 174
 in neural network learning, 123

P

PAC learning, 203–207, 225, 226
 of boolean conjunctions, 211–212
 definition of, 206–207
 training error in, 205
 true error in, 204–205
Paired tests, 147–150, 152
Parallelization in genetic algorithms, 268
Partially learned concepts, 38–39
Partially observable states in reinforcement
 learning, 369–370
Perceptron training rule, 88–89, 94, 95
Perceptrons, 86, 95, 96, 123
 representation of boolean functions,
 87–88

VC dimension of, 219
 weight update rule, 88–89, 94, 95
Perfect domain theory, 312–313
Performance measure, 6
Performance system, 11–12, 13
Philosophy, influence on machine
 learning, 4
Planning problems:
 PRODIGY in, 327
 case-based reasoning in, 240–241
Policy for selecting actions, 370–372
Population evolution, in genetic algorithms,
 260–262
Positive literal, 284, 285
Post-pruning:
 in decision tree learning, 68–69, 77,
 281
 in FOIL algorithm, 291
 in LEARN-ONE-RULE, 281
Posterior probability, 155–156, 162
Power law of practice, 4
Power set, 40–42
Predicates, 284, 285
Preference bias, 64, 76, 77
Prior knowledge, 155–156, 336. *See also*
 Domain theory
 to augment search operators, 357–361
 in Bayesian learning, 155
 derivatives of target function, 346–356,
 362
 in explanation-based learning,
 308–309
 explicit, use in learning, 329
 in human learning, 330
 initialize-the-hypothesis approach,
 339–346, 362
 in PROLOG-EBG, 313
 search alteration in inductive-analytical
 learning, 339–340, 362
 weighting in inductive-analytical
 learning, 338, 362
Prioritized sweeping, 380
Probabilistic reasoning, 163
Probabilities:
 estimation of, 179–180
 formulas, 159
 maximum likelihood (ML) hypothesis
 for prediction of, 167–170
Probability density, 165

Probability distribution, 133. *See also*
 Binomial distribution; Normal
 distribution
 Probably approximately correct (PAC)
 learning. *See* PAC learning
 Process control in manufacturing, 17
 PRODIGY, 326–327, 330
 Product rule, 159
 PROGOL, 300–302
 PROLOG, 275, 302, 330
 PROLOG-EBG, 313–321, 328–329
 applications of, 325
 deductive learning in, 321–322
 definition of, 314
 derivation of new features in, 320–321
 domain theory in, 322
 EBNN algorithm, comparison with, 356
 explanation of training examples,
 314–318
 weakest preimage in, 329
 inductive bias in, 322–323
 inductive logic programming,
 comparison with, 322
 limitations of, 329
 perfect domain theory in, 313
 prior knowledge in, 313
 properties of, 319
 regression process in, 316–318
 Propositional rules:
 learning by sequential covering
 algorithms, 275
 learning first-order rules, comparison
 with, 283
 Psychology, influence on machine
 learning, 4

Q

Q function:
 in deterministic environments, 374
 convergence of *Q* learning towards,
 377–380
 in nondeterministic environments, 381
 convergence of *Q* learning towards,
 382
Q learning algorithm, 372–376. *See also*
 Reinforcement learning
 advantages of, 386
 in deterministic environments, 375

 convergence, 377–380
 training rule, 375–376
 experimentation strategies in, 379
 lookup table, neural network substitution
 for, 384
 in nondeterministic environments,
 381–383
 convergence, 382–383
 training rule, 382
 updating sequence, 379
 Query strategies, 37–38

R

Radial basis function networks, 231,
 238–240, 245, 246, 247
 advantages of, 240
 Random variable, 133, 134, 137, 151
 Randomized method, 150
 Rank selection, 256
 RBF networks. *See* Radial basis function
 networks
 RDT program, 303
 Real-valued target function. *See*
 Continuous-valued target function
 Recurrent networks, 119–121. *See also*
 Neural networks, artificial
 Recursive rules, 284
 learning by FOIL algorithm, 290
 Reduced-error pruning, in decision tree
 learning, 69–71
 REGRESS algorithm, 317–318
 Regression, 236
 in PROLOG-EBG, 316–381
 Reinforcement learning, 367–387. *See also*
 Q learning algorithm
 applications of, 387
 differences from other methods, 369–370
 dynamic programming and, 380,
 385–387
 explanation-based learning and, 330
 function approximation algorithms in,
 384–385
 Relational descriptions, learning of, 302
 Relative frequency, 282
 Relative mistake bound for
 WEIGHTED-MAJORITY algorithm,
 224–225
 Residual, 236

Resolution rule, 293–294
 first-order, 296–297
 inverse entailment operator and, 294–296
 propositional, 294
 Restriction bias, 64
 Reward function, in reinforcement learning, 368
 Robot control:
 by BACKPROPAGATION and EBNN algorithms, comparison of, 356
 genetic programming in, 269
 Robot driving. *See* Autonomous vehicles
 Robot perception, attribute cost measures in, 76
 Robot planning problems, explanation-based learning in, 327
 ROTE-LEARNER algorithm, inductive bias of, 44–45
 Roulette wheel selection, 255
 Rule for estimating training values, 10, 383
 Rule learning, 274–303
 in decision trees, 71–72
 in explanation-based learning, 311–319
 by FOCL algorithm, 357–360
 by genetic algorithms, 256–259, 269–270, 274
 Rule post-pruning, in decision tree learning, 71–72
 Rules:
 disjunctive sets of, learning by sequential covering algorithms, 275–276
 first-order. *See* First-order rules
 propositional. *See* Propositional rules

S

SafeToStack, 310–312
 Sample complexity, 202. *See also* Training examples
 bound for consistent learners, 207–210, 225
 equation for, 209
 for finite hypothesis spaces, 207–214
 for infinite hypothesis spaces, 214–220
 of k -term CNF and DNF expressions, 213–214
 of unbiased concepts, 212–213
 VC dimension bound, 217–218

Sample error, 130–131, 133–134, 143
 training error and, 205
 Sampling theory, 132–141
 Scheduling problems:
 case-based reasoning in, 241
 explanation-based learning in, 325
 PRODIGY in, 327
 reinforcement learning in, 368
 Schema theorem, 260–262
 genetic operators in, 261–262
 Search bias. *See* Preference bias
 Search control problems:
 explanation-based learning in, 325–328, 329, 330
 limitations of, 327–328
 as sequential control processes, 369
 Search of hypothesis space. *See* Hypothesis space search
 Sequential control processes, 368–369
 learning task in, 370–373
 search control problems in, 369
 Sequential covering algorithms, 274, 275–279, 301, 313, 363
 choice of attribute-pairs in, 280–282
 definition of, 276
 FOIL algorithm, comparison with, 287, 301–302
 ID3 algorithm, comparison with, 280–281
 simultaneous covering algorithms, comparison with, 280–282
 variations of, 279–280, 286
 Shattering, 214–215
 Shepard's method, 234
 Sigmoid function, 97, 104
 Sigmoid units, 95–96, 115
 Simultaneous covering algorithms:
 choice of attributes in, 280–281
 sequential covering algorithms, comparison with, 280–282
 Single-point crossover operator, 254, 261
 SOAR, 327, 330
 Specific-to-general search, 281
 in FOIL algorithm, 287
 Speech recognition, 3
 BACKPROPAGATION algorithm in, 81
 representation by multilayer network, 95, 96
 weight sharing in, 118

- Speedup learning, 325, 330
 - SPHINX, 3
 - Split information, 73–74
 - Squashing function, 96
 - Stacking problems. *See also* SafeToStack
 - analytical learning in, 310
 - explanation-based learning in, 310
 - genetic programming in, 263–265
 - PRODIGY in, 327
 - Standard deviation, 133, 136–137
 - State-transition function, 380
 - Statistics:
 - basic definitions, 133
 - influence on machine learning, 4
 - Stochastic gradient descent, 93–94, 98–100, 104–105
 - Student t tests, 147–150, 152
 - Substitution, 285, 296
 - Sum rule, 159
- T**
- t tests, 147–150, 152
 - TANGENTPROP algorithm, 347–350, 362
 - BACKPROPAGATION algorithm, comparison with, 349
 - in EBNN algorithm, 352
 - search of hypothesis space
 - by KBANN and BACKPROPAGATION algorithms, comparison with, 350–351
 - tanh function, 97
 - Target concept, 22–23, 40–41
 - PAC learning of, 211–213
 - Target function, 7–8, 17
 - continuous-valued. *See* Continuous-valued target function
 - representation of, 8–9, 14, 17
 - TD-GAMMON, 3, 14, 369, 383
 - TD(λ) and BACKPROPAGATION algorithm in, 384
 - TD(λ), 383–384, 387
 - Temporal credit assignment, in reinforcement learning, 369
 - Temporal difference learning, 383–384, 386–387
 - Terms, in logic, 284, 285
 - Text classification, naive Bayes classifier in, 180–184
 - Theorem of total probability, 159
 - θ -subsumption, 302
 - relationship with entailment and more-general-than partial ordering, 299–300
 - Tournament selection, 256
 - Training and validation set approach, 69. *See also* Validation set
 - Training derivatives, 117–118
 - Training error:
 - of continuous-valued hypotheses, 89–90
 - of discrete-valued hypotheses, 205
 - in multilayer networks, 98
 - alternative error functions, 117–118
 - Training examples, 5–6, 17, 23. *See also* Sample complexity
 - explanation in PROLOG-EBG, 314–318
 - in PAC learning, 205–207
 - bounds on, 226
 - Voronoi diagram of, 233
 - Training experience, 5–6, 17
 - Training values, rule for estimation of, 10
 - True error, 130–131, 133, 137, 150, 204–205
 - of two hypotheses, differences in, 143–144
 - in version spaces, 208–209
 - Two-point crossover operator, 255, 257–258
 - Two-sided bounds, 141
- U**
- Unbiased estimator, 133, 137
 - Unbiased learners, 40–42
 - sample complexity of, 212–213
 - Uniform crossover operator, 255
 - Unifying substitution, 285, 296
 - Unsupervised learning, 191
 - Utility analysis, in explanation-based learning, 327–328
- V**
- Validation set. *See also* Training and validation set approach
 - cross-validation and, 111–112
 - error over, 110
 - Vapnik-Chervonenkis (VC) dimension. *See* VC dimension

Variables, in logic, 284, 285
 Variance, 133, 136–137, 138, 143
 VC dimension, 214–217, 226
 bound on sample complexity, 217–218
 definition of, 215
 of neural networks, 218–220
 Version space representation theorem, 32
 Version spaces, 29–39, 46, 47, 207–208
 Bayes optimal classifier and, 176
 definition of, 30
 exhaustion of, 208–210, 226
 representations of, 30–32
 Voronoi diagram, 233

W

Weakest preimage, 316, 329
 Weight decay, 111, 117
 Weight sharing, 118

Weight update rules, 10–11
 BACKPROPAGATION weight update rule,
 101–103
 alternative error functions, 117–118
 in KBANN algorithm, 343–344
 optimization methods, 119
 output units, 171
 delta rule, 11, 88–90, 94
 gradient ascent, 170–171
 gradient descent, 91–92, 95
 linear programming, 95
 perceptron training rule, 88–89
 stochastic gradient descent, 93–94
 WEIGHTED-MAJORITY algorithm, 222–226
 mistake-bound learning in, 224–225
 Weighted voting, 222, 223, 226
 Widrow-Hoff rule. *See* Delta rule